



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق
پاسخ تمرین شماره یک

دانشجویان:

علی رجب زاده ۸۱۰۱۰۴۱۳۷

رضا مرید شوشتری ۸۱۰۱۰۴۲۵۰

فهرست مطالب

۴	فهرست تصاویر
۶	فهرست جداول
۷	۱ سوال اول: نورون McCulloch-Pitts
۷	۱-۱ منطق آستانه‌ای
۷	۱-۱-۱ پیاده‌سازی گیت AND
۷	۱-۱-۲ پیاده‌سازی گیت OR
۸	۱-۱-۳ پیاده‌سازی گیت NOT
۹	۱-۱-۴ ناتوانی نورون McCulloch-Pitts در رسم مرزهای غیرخطی
۹	۱-۲ Majority و Parity
	۱-۲-۱ توضیح هندسی درباره غیرقابل‌نمایش بودن تابع XOR سه‌تایی با نورون McCulloch-Pitts
۹	۱-۲-۲ طراحی کوچکترین شبکه دولایه آستانه‌ای برای تابع XOR سه‌تایی
۱۱	۱-۲-۳ طراحی کوچکترین شبکه دولایه آستانه‌ای برای تابع majority پنج‌تایی
۱۳	
۱۵	۲ سوال دوم : Madaline and Adaline
۱۵	۱-۲ Adaline روی Cancer Breast
۱۵	۱-۲-۱ از SSE تا LMS
۱۶	۱-۲-۲ آماده‌سازی داده و تحلیل اولیه
۱۶	۱-۲-۳ پیاده‌سازی Adaline از صفر
۱۷	۱-۲-۴ ارزیابی و مصور سازی
۲۱	۱-۲-۵ مدل‌های پایه و بحث
۲۲	۲-۲ Madaline Rull II روی داده‌های Circle و Moon
۲۲	۱-۲-۲ پیاده‌سازی Madaline با قانون دوم
۲۴	۲-۲-۲ داده‌ها و تنظیمات آموزشی

۲۴		ارزیابی و نمودار ها	۳-۲-۲
۳۳		مطالعه ابر پارامتر ها	۴-۲-۲
۳۴		۳ سوال سوم: رگرشن با استفاده از mlp	
۳۴		دریافت دیتاست و انجام پیش پردازش های لازم	۱-۳
۳۴		مدیریت داده های گمشده	۱-۱-۳
۳۴		حذف داده های پرت از قسمت ستون هدف	۲-۱-۳
۳۵		تبدیل فیچرهای categorical به یک مقدار قابل قبول	۳-۱-۳
۳۶		پیداسازی یک شبکه عصبی ساده	۲-۳
۳۷		آموزش شبکه عصبی ساده	۳-۳
۳۹		طراحی و آموزش شبکه عصبی پیچیده تر	۴-۳
۴۱		مقایسه	۵-۳
۴۴		۴ طراحی شبکه Auto Encoder	
۴۴		دریافت دیتاست و انجام پیش پردازش های لازم	۱-۴
۴۴		پیداسازی	۲-۴
۴۵		آموزش شبکه	۳-۴
۴۸		ارزیابی شبکه	۴-۴
۴۸		انکود کردن داده ها	۱-۴-۴
۴۸		کلاسترینگ با K-means	۲-۴-۴
۴۸		تحلیل خوشه ها	۳-۴-۴
۵۰		کاهش ابعاد و t-SNE	۴-۴-۴
۵۰		مقایسه با داده های خام	۵-۴-۴
۵۲		ارزیابی کیفیت خوشه بندی	۶-۴-۴

فهرست تصاویر

۱۰	نمایش نقاط ورودی به XOR سه‌تایی و خروجی مربوط به آنها	۱
۲	شبکه دولایه مینیمال برای تابع XOR سه‌تایی؛ لایه پنهان شامل سه آستانه شمارشی H_1, H_2, H_3	۲
۱۲	است و نوروں خروجی ترکیب $H_1 - H_2 + H_3$ را می‌سنجد.	۱۲
۳	شبکه دولایه مینیمال برای تابع اکثریت با پنج ورودی. نوروں پنهان شرط $\sum_{i=1}^5 x_i \geq 3$ را پیاده‌سازی می‌کند. لایه خروجی، به‌عنوان یک لایه همانی عمل می‌کند و تنها خروجی لایه پنهان را به بیرون می‌فرستد.	۱۴
۴	مقادیر SSE و Accuracy در هر epoch	۱۷
۵	ماتریس درهم‌ریختگی	۱۸
۶	2D test dataset	۱۹
۷	مرز تصمیم آدالین روی داده‌های تست	۲۰
۸	False negative and False Positive	۲۱
۹	Moon and Circle dataset	۲۴
۱۰	SSE and Accuracy (L stand for L2 regularization)	۲۵
۱۱	ماتریس درهم‌ریختگی	۲۶
۱۲	dataset Moon for boundaries decision Madaline	۲۷
۱۳	Madaline decision boundaries with 20 neurons	۲۸
۱۴	Madaline decision boundaries for Circle dataset	۲۹
۱۵	SSE and Accuracy during training	۳۰
۱۶	ماتریس درهم‌ریختگی	۳۱
۱۷	Madaline decision boundaries for Circle dataset	۳۲
۱۸	تعداد داده‌های گمشده مربوط به هر ستون دیتاست	۳۴
۱۹	نمودار جعبه‌ای و نمودار هیستوگرام داده‌های ستون هدف	۳۵
۲۰	نمایش تعداد ریکوردها قبل و بعد از حذف داده‌های پرت	۳۵
۲۱	نمایش ساختار و تعداد پارامترهای شبکه عصبی ساده پیشنهادی	۳۶

۲۲	نمودار loss برای مجموعه داده های آموزش و اعتبارسنجی در فاز آموزش شبکه عصبی ساده	۳۹
۲۳	نمایش ساختار و تعداد پارامترهای شبکه عصبی پیچیده تر انتخاب شده	۴۰
۲۴	نمودار loss برای مجموعه داده های آموزش و اعتبارسنجی در فاز آموزش شبکه عصبی پیچیده تر	۴۱
۲۵	نمودار Actual vs Predicted برای شبکه عصبی پیچیده تر	۴۲
۲۶	MSE over Epochs	۴۵
۲۷	Ankle Boot class	۴۶
۲۸	Trouser class	۴۶
۲۹	Bag class	۴۷
۳۰	Pullover class	۴۷
۳۱	Sandal class	۴۸
۳۲	ماتریس درهم ریختگی	۴۹
۳۳	2D test dataset	۵۰
۳۴	ماتریس درهم ریختگی	۵۱
۳۵	2D test dataset(without Encoding)	۵۲

فهرست جداول

۷	جدول درستی گیت AND با استفاده از نورون McCulloch–Pitts	۱
۸	جدول درستی گیت OR با استفاده از نورون McCulloch–Pitts	۲
۸	جدول درستی گیت NOT با استفاده از نورون McCulloch–Pitts	۳
۱۲	مقادیر داخلی برای تمام حالات (x_1, x_2, x_3)	۴
۱۹	ارزیابی مدل	۵
۲۵	دقت مدل ها	۶
۲۶	ارزیابی مدل	۷
۲۹	دقت مدل ها	۸
۳۰	دقت مدل ها	۹
۳۱	ارزیابی مدل	۱۰
۳۳	خلاصه نتایج برای Seed صفر	۱۱
۳۳	خلاصه نتایج برای Seed یک	۱۲
۳۳	خلاصه نتایج برای Seed دو	۱۳
۳۸	مقایسه‌ی معیارهای ارزیابی مدل برای مدل شبکه عصبی ساده	۱۴
۴۱	مقایسه‌ی معیارهای ارزیابی مدل برای مدل شبکه عصبی پیچیده‌تر	۱۵
۴۹	کلاس‌های دیتاست Fashion-MNIST	۱۶
۵۲	datasets test encoded and raw for metrics clustering of Comparison	۱۷

۱ سوال اول: نورون McCulloch–Pitts

۱-۱ منطق آستانه‌ای

۱-۱-۱ پیاده‌سازی گیت AND

در گیت منطقی AND، خروجی تنها زمانی برابر ۱ است که هر دو ورودی برابر با ۱ باشند. در این حالت وزن‌ها و آستانه به صورت زیر انتخاب می‌شوند:

$$w_1 = 1, \quad w_2 = 1, \quad \theta = 2$$

فرمول عملکرد نورون به صورت زیر خواهد بود:

$$y = \begin{cases} 1, & \text{اگر } x_1 + x_2 \geq 2 \\ 0, & \text{در غیر این صورت} \end{cases}$$

جدول ۱: جدول درستی گیت AND با استفاده از نورون McCulloch–Pitts

خروجی (y)	x_2	x_1
۰	۰	۰
۰	۱	۰
۰	۰	۱
۱	۱	۱

۲-۱-۱ پیاده‌سازی گیت OR

در گیت منطقی OR، خروجی برابر ۱ است اگر حداقل یکی از ورودی‌ها برابر ۱ باشد. در این حالت وزن‌ها و آستانه به صورت زیر انتخاب می‌شوند:

$$w_1 = 1, \quad w_2 = 1, \quad \theta = 1$$

$$y = \begin{cases} 1, & \text{اگر } x_1 + x_2 \geq 1 \\ 0, & \text{در غیر این صورت} \end{cases}$$

جدول ۲: جدول درستی گیت OR با استفاده از نورون McCulloch–Pitts

x_1	x_2	خروجی (y)
۰	۰	۰
۱	۰	۱
۰	۱	۱
۱	۱	۱

۳-۱-۱ پیاده‌سازی گیت NOT

در گیت منطقی NOT تنها یک ورودی وجود دارد و خروجی زمانی برابر ۱ است که ورودی برابر ۰ باشد. در این حالت پارامترها به صورت زیر در نظر گرفته می‌شوند:

$$w = -1, \quad \theta = 0$$

$$y = \begin{cases} 1, & \text{اگر } -x_1 \geq 0 \\ 0, & \text{در غیر این صورت} \end{cases}$$

جدول ۳: جدول درستی گیت NOT با استفاده از نورون McCulloch–Pitts

x_1	خروجی (y)
۰	۱
۱	۰

در هر کدام از گیت‌ها به کمک جدول درستی و جاگذاری هر کدام از سطور به راحتی می‌توان درستی بایاس و وزن‌های انتخاب‌شده را صحت‌سنجی کرد.

۴-۱-۱ ناتوانی نورون McCulloch–Pitts در رسم مرزهای غیرخطی

در مدل McCulloch–Pitts، تصمیم‌گیری نورون بر اساس مقایسه یک مجموع وزنی از ورودی‌ها با آستانه θ انجام می‌شود. به بیان ریاضی، مرز تصمیم‌گیری این نورون توسط معادله زیر تعیین می‌شود:

$$w_1x_1 + w_2x_2 + \dots + w_nx_n = \theta$$

این معادله نشان‌دهنده یک ابرصفحه خطی در فضای ورودی است (در حالت دوبعدی، یک خط مستقیم). ورودی‌هایی که در یک سمت این مرز قرار دارند، خروجی ۱ تولید می‌کنند و ورودی‌هایی که در سمت دیگر قرار دارند، خروجی ۰ خواهند داشت.

به صورت شهودی، این یعنی نورون McCulloch–Pitts تنها قادر است مجموعه داده‌هایی را از هم جدا کند که بتوان آن‌ها را با یک خط (یا صفحه) جداکننده از هم تفکیک کرد. به چنین توابعی، توابع خطی قابل جداسازی (Linearly Separable) گفته می‌شود.

به عنوان مثال در تابع‌های AND و OR، می‌توان نقاط متناظر با خروجی ۱ و ۰ را در صفحه (x_1, x_2) با یک خط جدا کرد. از همین نظرگاه، می‌توان ناتوانی چنین مدلی را در تشخیص مرزهای غیرخطی نتیجه گرفت. در استدلال مشابه، به دلیل اینکه مرز پیشنهادی این مدل، کاملاً خطی است، در مسائلی که با مرزهای غیرخطی روبه‌رو می‌شویم، این مدل ناتوان خواهد بود.

۲-۱ Majority و Parity

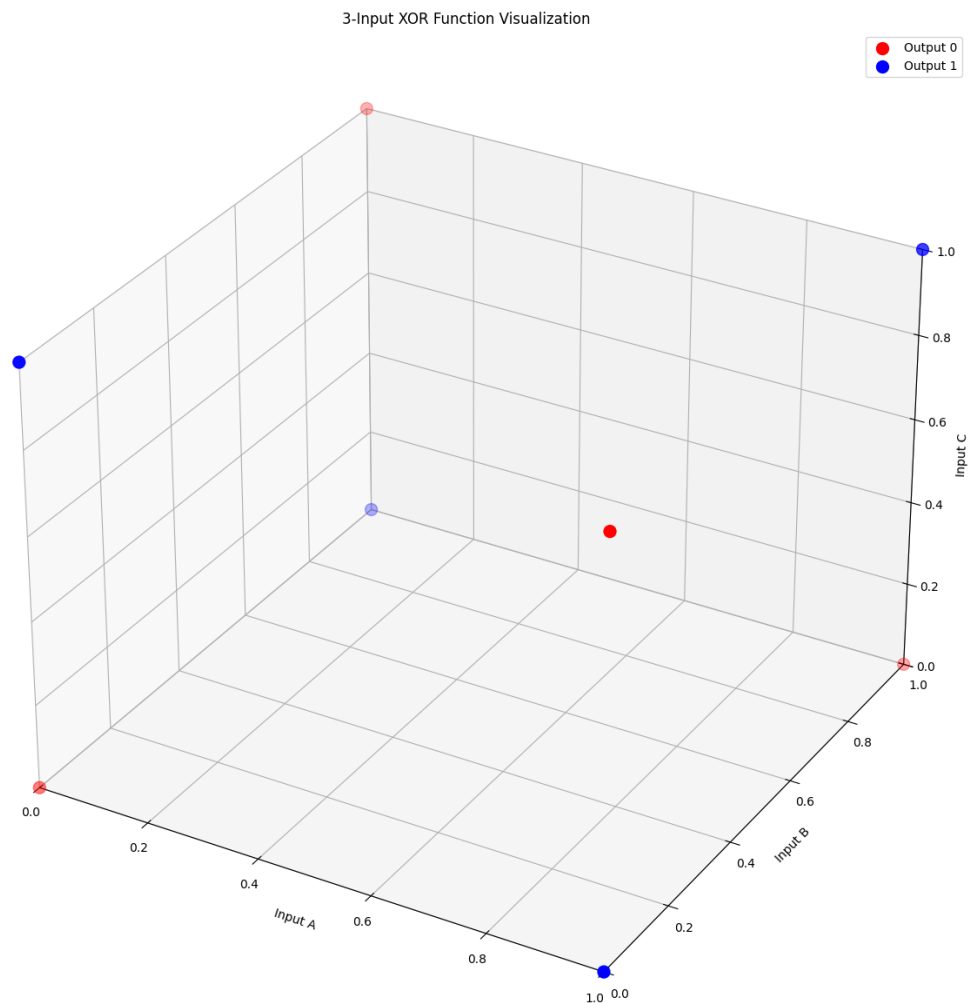
۱-۲-۱ توضیح هندسی درباره غیرقابل‌نمایش بودن تابع XOR سه‌تایی با نورون McCulloch–Pitts

در تابع XOR سه‌تایی، خروجی زمانی برابر ۱ است که تعداد ورودی‌های برابر با ۱، فرد باشد (یعنی یکی یا سه تا از آن‌ها فعال باشند). اگر نقاط ورودی را در فضای سه‌بعدی (x_1, x_2, x_3) در نظر بگیریم، هشت نقطه ممکن رأس‌های یک مکعب را تشکیل می‌دهند. نقاطی که خروجی ۱ دارند (جمع فرد از ۱‌ها) و نقاطی که خروجی ۰ دارند (جمع زوج از ۱‌ها) به گونه‌ای در این مکعب قرار گرفته‌اند که هیچ صفحه خطی نمی‌تواند آن‌ها را به طور هم‌زمان از هم جدا کند.

به بیان دیگر، مرز تصمیم‌گیری نورون McCulloch–Pitts که به صورت یک صفحه خطی تعریف می‌شود،

۱ سوال اول: نورون McCulloch–Pitts

قادر نیست نقاط با جمع فرد را در یک سمت و نقاط با جمع زوج را در سمت دیگر قرار دهد؛ زیرا این نقاط به صورت متقارن در فضای ورودی پراکنده‌اند. بنابراین، تابع XOR سه‌تایی غیرخطی بوده و با یک نورون از این نوع قابل نمایش نیست. همچنین در قسمت زیر شکل قرارگیری این ۸ نقطه در فضا نمایش داده شده است. همان طور که مشخص است، نمیتوان با یک تک خط، دو دسته نقاط را از یکدیگر جدا کرد.



شکل ۱: نمایش نقاط ورودی به XOR سه‌تایی و خروجی مربوط به آنها

۱-۲-۲ طراحی کوچک‌ترین شبکه دولایه آستانه‌ای برای تابع XOR سه‌تایی

از سه نورون در لایه پنهان استفاده می‌کنیم که هرکدام یک آستانه شمارشی روی مجموع ورودی‌ها پیاده می‌کنند:

$$H_1(x) = 1\{x_1 + x_2 + x_3 \geq 1\}, \quad H_2(x) = 1\{x_1 + x_2 + x_3 \geq 2\}, \quad H_3(x) = 1\{x_1 + x_2 + x_3 \geq 3\}.$$

نورون خروجی را طوری می‌گیریم که فعال شود اگر

$$H_1 - H_2 + H_3 \geq 1.$$

همان‌طور که بررسی می‌شود، برای مجموع $s = 0, 1, 2, 3$ این مقدار برابر $0, 1, 0, 1$ شده و دقیقاً parity را می‌دهد. برای سادگی همه وزن‌های ورودی به نورون‌های پنهان را ۱ می‌گیریم:

$$H_1 : \quad w = [1, 1, 1], \quad b = -1 \quad (\text{active } s \geq 1)$$

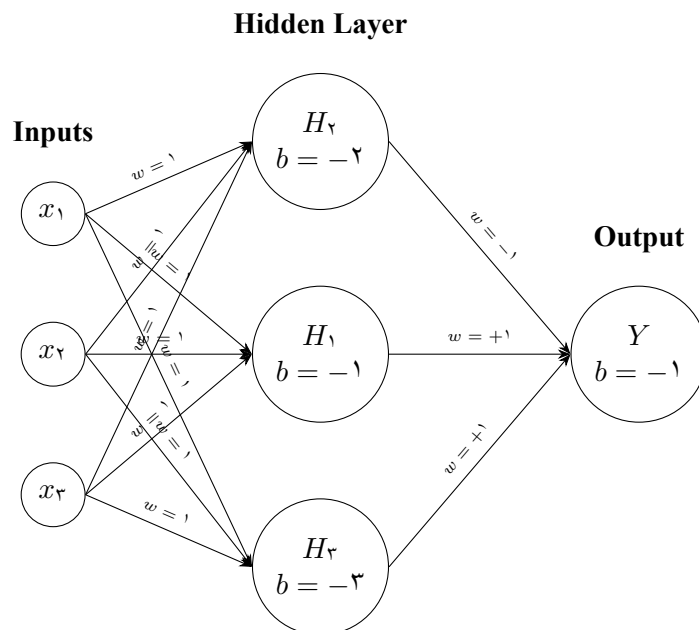
$$H_2 : \quad w = [1, 1, 1], \quad b = -2 \quad (\text{active } s \geq 2)$$

$$H_3 : \quad w = [1, 1, 1], \quad b = -3 \quad (\text{active } s \geq 3)$$

نورون خروجی:

$$Y : \quad w = [1, -1, 1] \text{ for } [H_1, H_2, H_3], \quad b = -1.$$

قاعده فعال‌سازی همه نورون‌ها: فعال اگر $\sum w_i x_i + b \geq 0$. همچنین در شکل زیر، معماری پیشنهادی نمایش داده شده است.



شکل ۲: شبکه دو لایه مینیمال برای تابع XOR سه‌تایی؛ لایه پنهان شامل سه آستانه شمارشی H_1, H_2, H_3 است و نورون خروجی ترکیب $H_1 - H_2 + H_3$ را می‌سازد.

همچنین جدول عملکرد این معماری، به ازای تمامی ورودی‌های ممکن در قسمت زیر آمده است.

جدول ۴: مقادیر داخلی برای تمام حالات (x_1, x_2, x_3)

x_1	x_2	x_3	H_1	H_2	H_3	$\text{pre}_Y = H_1 - H_2 + H_3$	Y
۰	۰	۰	۰	۰	۰	۰	۰
۰	۰	۱	۱	۰	۰	۱	۰
۰	۱	۰	۱	۰	۰	۱	۰
۰	۱	۱	۱	۱	۰	۰	۰
۱	۰	۰	۱	۰	۰	۱	۱
۱	۰	۱	۱	۱	۰	۰	۱
۱	۱	۰	۱	۱	۰	۰	۱
۱	۱	۱	۱	۱	۱	۱	۱

در جهت توضیح معماری انتخابی، نورون‌های پنهان H_1, H_2, H_3 به ترتیب رخداد «حداقل یک ۱»، «حداقل دو ۱» و «سه ۱» را تشخیص می‌دهند. ترکیب خطی $H_1 - H_2 + H_3$ مقدار ۱ را دقیقاً برای حالات با تعداد ۱‌های فرد (۱ یا ۳) می‌دهد، بنابراین یک نورون آستانه‌ای با بایاس -1 می‌تواند نتیجه XOR سه‌تایی را تولید کند. این طراحی دو لایه (سه نورون پنهان + یک خروجی) کمینه و کارا است.

۱-۲-۳ طراحی کوچکترین شبکه دولایه آستانه‌ای برای تابع majority پنج‌تایی

تابع Majority ۵ خروجی ۱ تولید می‌کند اگر حداقل ۳ عدد از ورودی‌های دودویی $x_1, \dots, x_5$ برابر ۱ باشند. این تابع را می‌توان با یک نورون McCulloch–Pitts واحد نمایش داد؛ بنابراین کوچکترین شبکه عملاً همان یک نورون است و تابع جدایی‌پذیر خطی می‌باشد. با این حال برای رعایت صورت سؤال، نسخه دولایه ساده‌ای نیز ارائه می‌دهیم که معادل همان نورون تکی است. انتخاب وزن‌ها و بایاس را به شکل زیر انجام می‌دهیم:

$$w_i = 1 \quad (i = 1, \dots, 5), \quad b = -3.$$

زیرا شرط

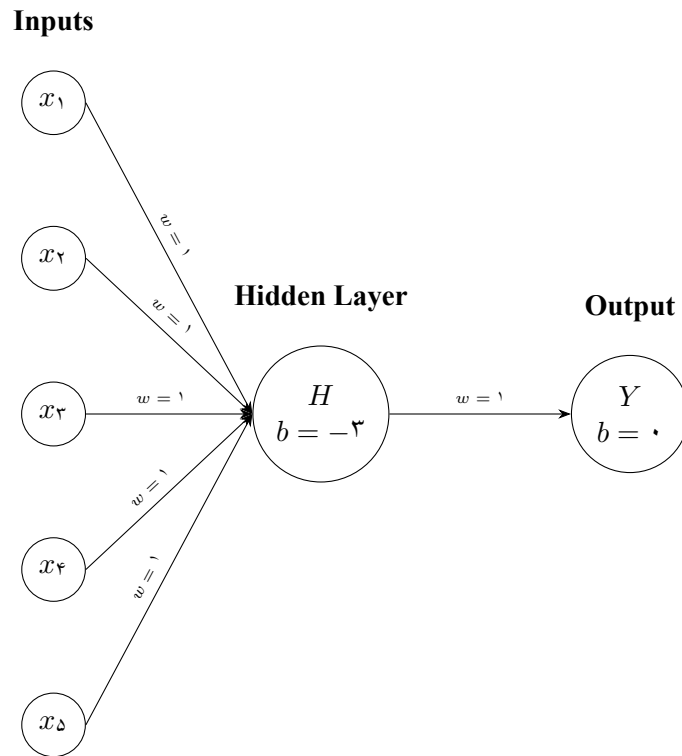
$$\sum_{i=1}^5 x_i - 3 \geq 0 \iff \sum_{i=1}^5 x_i \geq 3$$

دقیقاً تعریف تابع اکثریت را می‌دهد.

از لحاظ هندسی مرز تصمیم‌گیری این نورون را معادله $x_1 + x_2 + \dots + x_5 = 3$ تعیین می‌کند که یک ابرصفحه خطی در فضای پنج‌بعدی است. همه نقاط که مجموعشان بزرگتر یا مساوی ۳ است در یک‌سو و بقیه در سوی دیگر قرار می‌گیرند؛ به این دلیل تابع خطی قابل جداسازی است. برای نمایش به صورت «دو لایه» به صورت زیر عمل می‌کنیم:

$$H : w = [1, 1, 1, 1, 1], b = -3, \quad Y : w_Y = [1], b_Y = 0.$$

یعنی H همان تصمیم‌گیر اصلی است و Y فقط خروجی H را پاس می‌دهد. این ساختار دو لایه، معادل تک‌نورون است ولی به فرم دو لایه نوشته شده است. در نتیجه نیازی به شبکه پیچیده با نورون‌های پنهان متعددی نیست؛ یک آستانه مناسب کافی است.



شکل ۳: شبکه دو لایه مینیمال برای تابع اکثریت با پنج ورودی. نورون پنهان شرط $\sum_{i=1}^5 x_i \geq 3$ را پیاده‌سازی می‌کند. لایه خروجی، به عنوان یک لایه همانی عمل می‌کند و تنها خروجی لایه پنهان را به بیرون می‌فرستد.

۲ سوال دوم : Madaline and Adaline

۱-۲ Adaline روی Cancer Breast

۱-۱-۲ از SSE تا LMS

از SSE میدانیم که:

$$SSE = \sum_{i=1}^N (y^{(i)} - \text{net}^{(i)})^2$$

در ادامه اگر از SSE نسبت به وزن ها مشتق بگیریم خواهیم داشت:

$$\frac{\partial SSE}{\partial w_j} = -2 \sum_{i=1}^N (y^{(i)} - \text{net}^{(i)}) x_j^{(i)}$$

حال مقدار وزن ها را طبق گرادینان آپدیت میکنیم و به قانون LMS میرسیم (η نرخ یادگیری است)

$$w_j + = 2\eta \sum_{i=1}^N (y^{(i)} - \text{net}^{(i)}) x_j^{(i)}$$

در روش بروزرسانی آنلاین وزن ها را به صورت تکی آپدیت میکنیم:

$$w_j + = 2\eta (y^{(i)} - \text{net}^{(i)}) x_j^{(i)}$$

اگر جریمه L_2 را به تابع SSE اضافه کنیم خواهیم داشت:

$$J(\mathbf{w}) = \sum_{i=1}^N (y^{(i)} - \mathbf{w} \cdot \mathbf{x}^{(i)})^2 + \lambda \sum_{j=1}^m w_j^2$$

۲ سوال دوم : Madaline and Adaline

و در ادامه از این تابع $J(w)$ مشتق میگیریم:

$$\frac{\partial J}{\partial w_j} = -\sum_{i=1}^N (y^{(i)} - \mathbf{w} \cdot \mathbf{x}^{(i)}) x_j^{(i)} + \lambda w_j$$

و وزن ها را با روش گرادیان آپدیت میکنیم:

$$w_j + = \eta \sum_{i=1}^N (y^{(i)} - \mathbf{w} \cdot \mathbf{x}^{(i)}) x_j^{(i)} - \eta \lambda w_j$$

که این همان روش regression ridge است.

۲-۱-۲ آماده سازی داده و تحلیل اولیه

در این قسمت داد ها را از کتابخانه Sklearn میخوانیم و با متود train_test_split یکبار ۳۰ درصد داده ها را برای تست و بار دیگر ۲۰ درصد ۷۰ درصد داده های باقی مانده را برای validation در طول آزمون بخش بندی میکنیم و با روش standard_scaler داده ها را نرمالایز میکنیم به طوری که میانگین هر فیچر صفر و انحراف معیار آن برابر با ۱ شود. همچنین با متود Stratify توازن کلاس ها را در داده های آموزش با داده های اصلی برابر کردیم. (۶۳ درصد برای کلاس مثبت و ۳۷ درصد برای کلاس منفی)

۳-۱-۲ پیاده سازی Adaline از صفر

در این بخش ما کلاس Adaline را طراحی میکنیم. توابع استفاده شده در کلاس به شرح زیر است:

- fit(): در این تابع داده های train و valid را و همچنین مقادیر اولیه وزن ها و بایاس را صفر ست میکنیم.

- train(): در این تابع با توجه به تعداد epoch ها و مقدار لاندا برای جریمه L_2 و با توجه به LMS وزن ها را آپدیت میکنیم. (روش بچ گرادیانت) همچنین بدلیل اینکه مقادیر net ممکن است مقادیر در روند آموزش اختیار کنند و روند آموزش را ناپایدار کنند باید مقدار گرادیانت را بر تعداد کل سمپل ها تقسیم کرد.

۲ سوال دوم : Madaline and Adaline

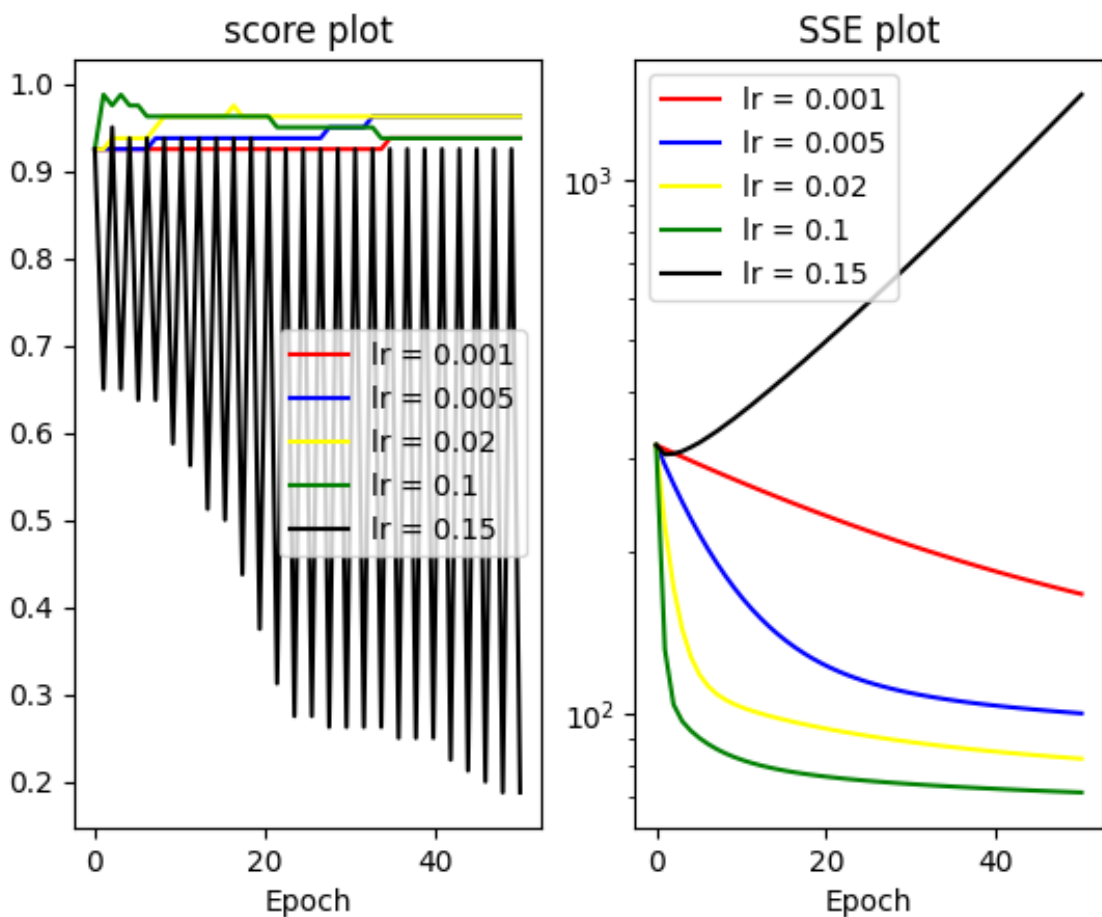
• `Online_train()` : تفاوت این تابع با تابع `train` در این است که بجای آپدیت کردن در آخر هر epoch بعد از محاسبه ارور هر سمپل ، وزن ها را آپدیت میکنیم.

• `predict()` : در این تابع مقدار `X_test` را میگیریم و `Y_test` را محاسبه میکنیم.

همچنین تابع کمکی `Eval()` را برای محاسبه دقت قرار داده ایم.

۴-۱-۲ ارزیابی و مصور سازی

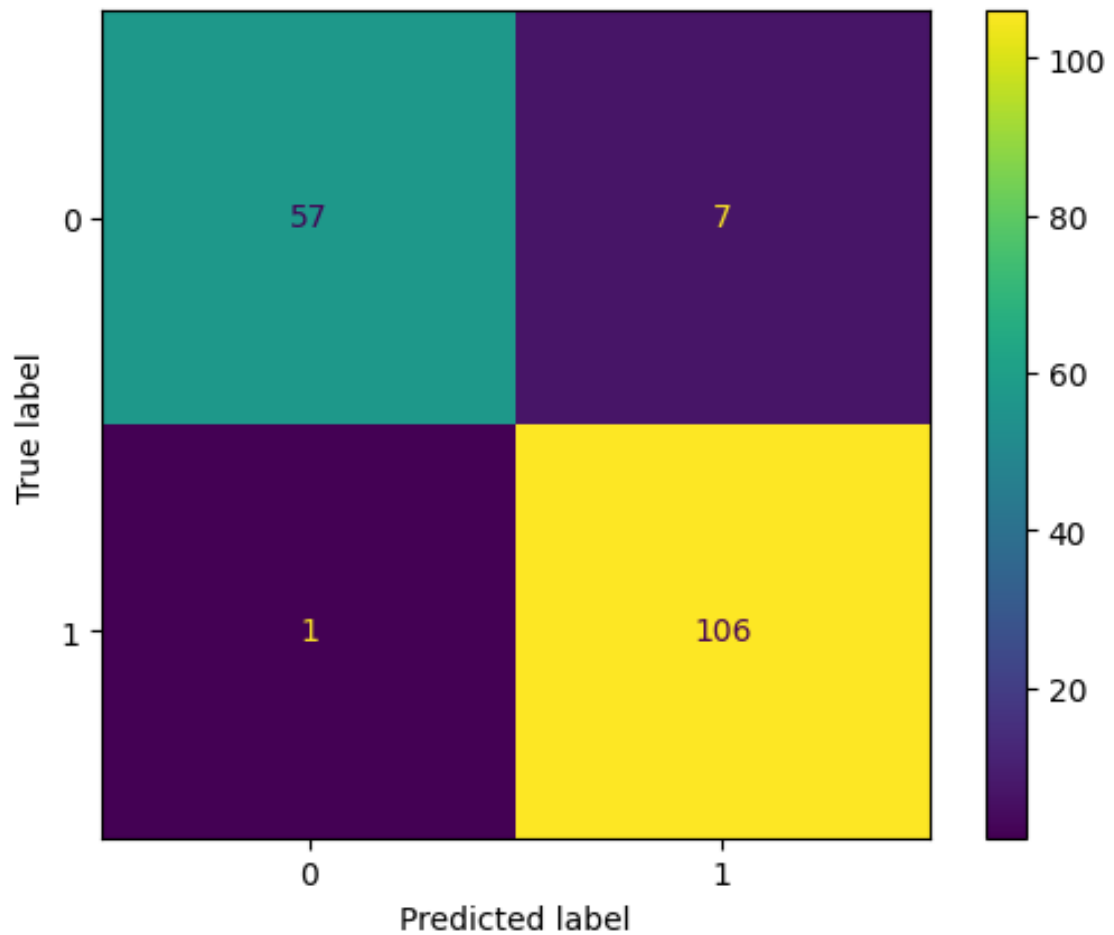
در ادامه مدل را با لرنینگ ریت های یک هزارم و پنج هزارم و دو صدم و یک دهم و پانزده صدم ترین کردیم و نتایج به صورت زیر بدست آمدند:



شکل ۴: مقادیر SSE و Accuracy در هر epoch

۲ سوال دوم : Madaline and Adaline

با افزایش نرخ یادگیری تا عدد یک دهم یادگیری مدل افزایش یافت و مقدار خطای SSE کاهش پیدا کرد ولی در نرخ پانزده صدم مدل ناپایدار شد و مقدار خطا افزایش یافت و مدل ناپایدار شد. همچنین با توجه به داده های تست بهترین مدل که مدل با نرخ یادگیری یک دهم باشد را به عنوان بهترین مدل انتخاب میکنیم و ارزیابی را با این مدل پیش میبریم. ابتدا ماتریس درهم ریختگی را برای این مدل رسم میکنیم:



شکل ۵: ماتریس درهم ریختگی

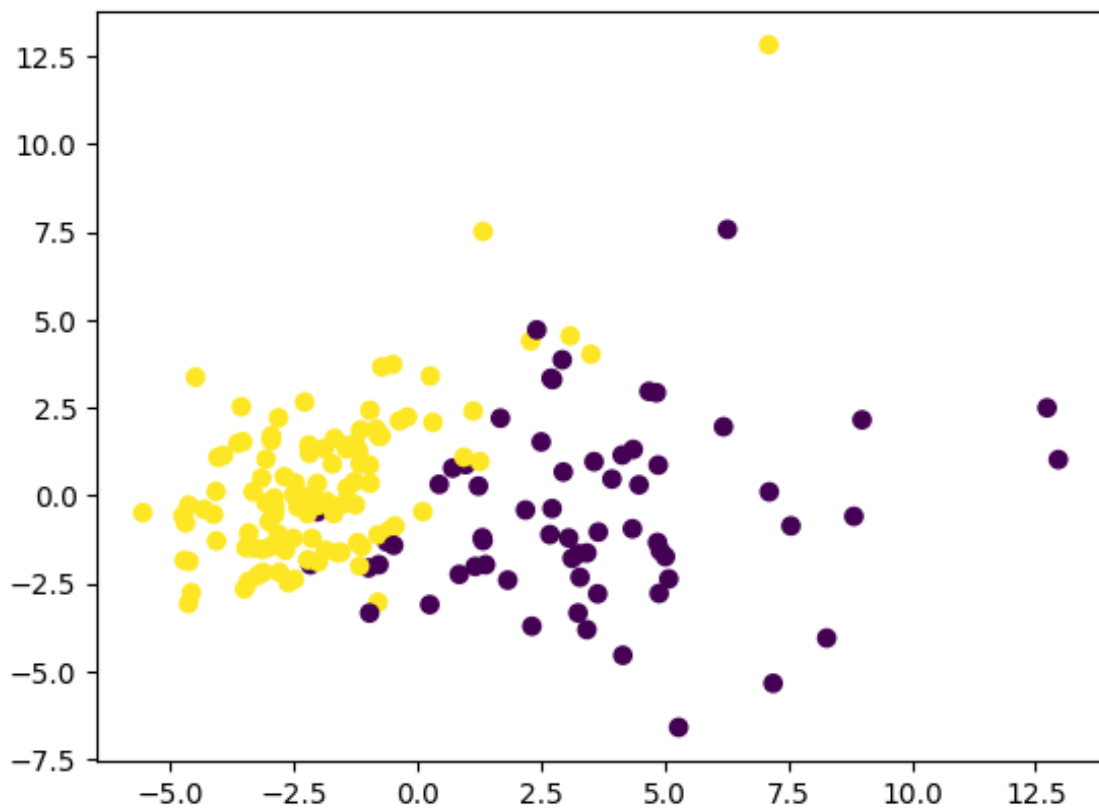
۲ سوال دوم : Madaline and Adaline

با توجه به این مقادیر می‌توان recall و precision را بدست آورد و همچنین با تابع sklearn مقدار ROC_AUC و F1_score را محاسبه می‌کنیم و مقادیر زیر را بدست می‌آوریم:

جدول ۵: ارزیابی مدل

ROC_AUC	F1_score	Recall	Precision
0.941	0.964	0.991	0.938

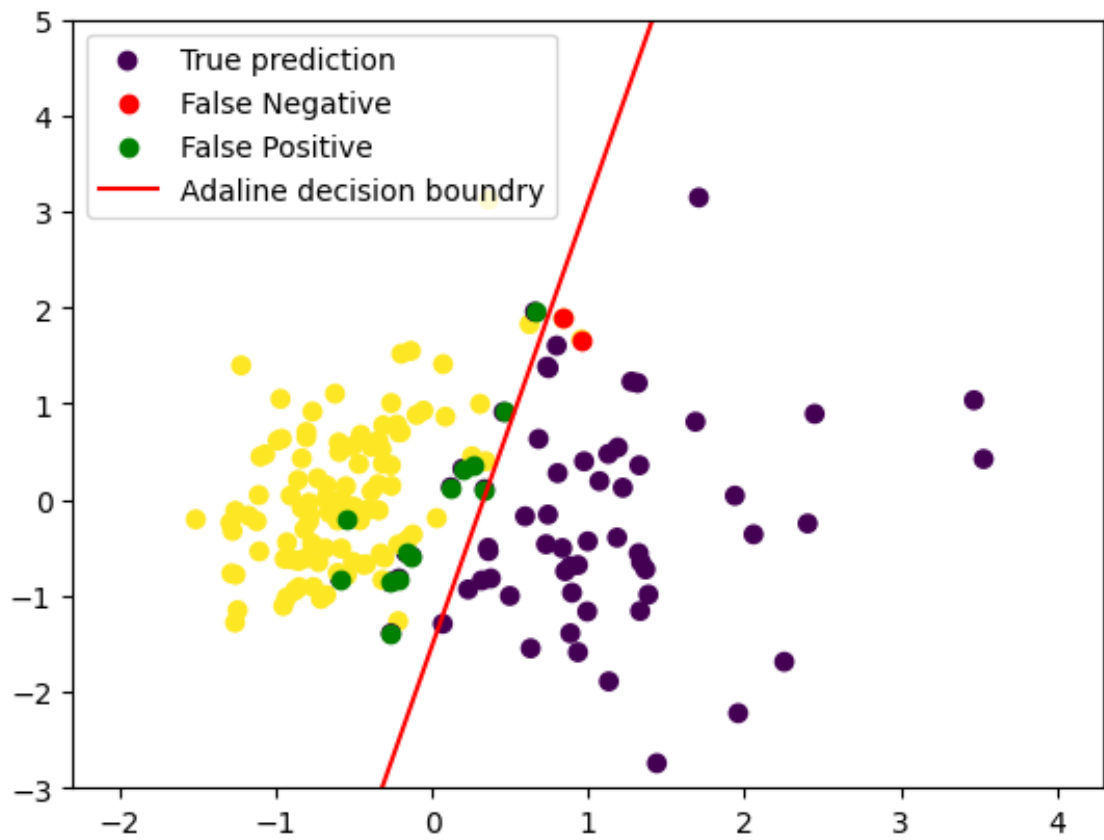
در ادامه داده های آزمون را با PCA به فضای ۲ بعدی می‌بریم و نمودار آن را رسم می‌کنیم.



شکل ۶: 2D test dataset

حال مدل را با داده های آموزش دو بعدی شده آموزش می‌دهیم و وزن و بایاس آن را بدست می‌آوریم و مرز تصمیم را در این فضای دوبعدی رسم می‌کنیم:

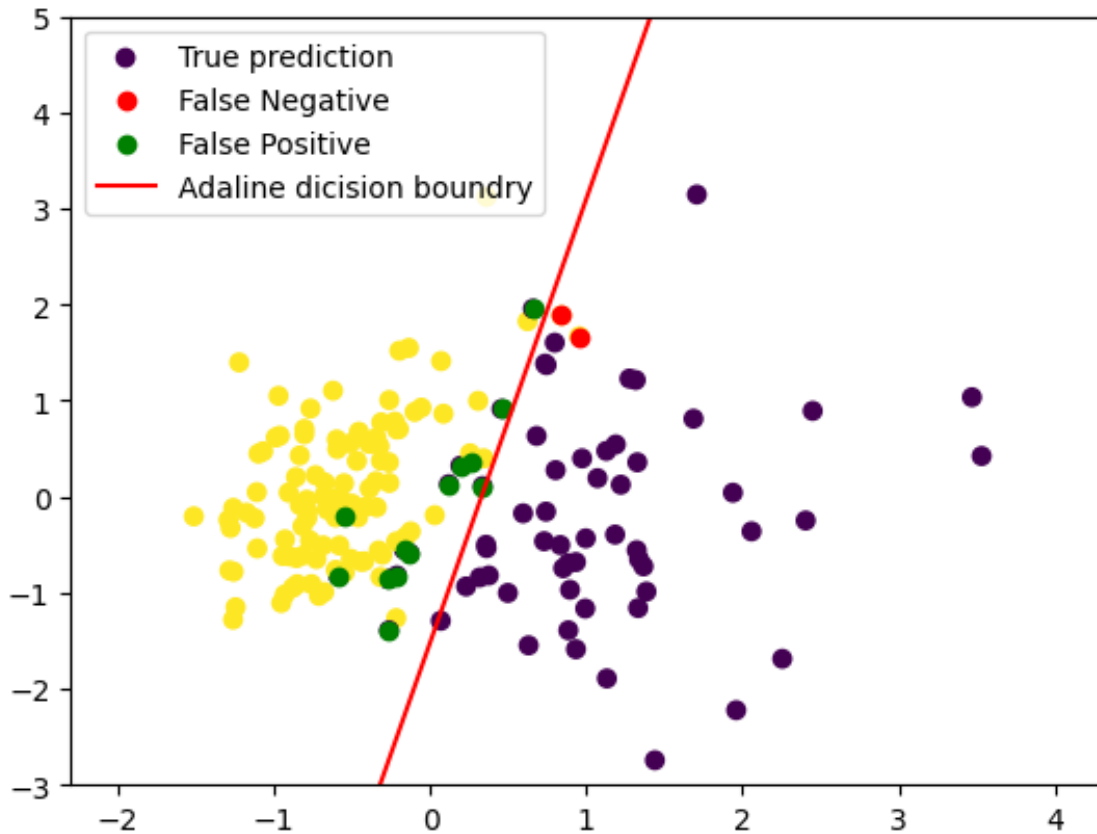
۲ سوال دوم : Madaline and Adaline



شکل ۷: مرز تصمیم آدالین روی داده های تست

۲ سوال دوم : Madaline and Adaline

روی نمودار زیر می‌توان داده‌های False Negative و False Positive را مشاهده کرد:



شکل ۸: False negative and False Positive

۵-۱-۲ مدل‌های پایه و بحث

مدل Adaline مانند Logistic Regression یک تفکیک‌کننده خطی است، اما به جای تابع سیگموئید از تابع خطی و MSE برای آموزش استفاده می‌کند. به همین دلیل در داده‌هایی با نویز زیاد یا برجستگی‌های غیرخطی، Regression Logistic معمولاً عملکرد دقیق‌تری دارد چون از تابع هزینه $\log\text{-loss}$ و احتمال‌محور استفاده می‌کند. LDA هم در صورت نرمال بودن داده‌ها و برابری کوواریانس کلاس‌ها عملکردی مشابه یا بهتر از Adaline دارد. در مجموع، Adaline در مسائل ساده و خطی با داده‌های نرمال‌ایز شده نتایج خوبی می‌دهد، اما در داده‌های واقعی Logistic Regression و LDA پایدارتر و دقیق‌ترند.

۲-۲ Madaline Rull II روی داده های Moon و Circle

۱-۲-۲ پیاده سازی Madaline با قانون دوم

در این بخش ما کلاس Madaline با قانون دوم را پیاده سازی میکنیم و در ادامه یک شبهه کد از این کلاس را آورده ایم. در ابتدا توابع این کلاس را معرفی میکنیم:

- `fit ()` : مقادیر `train` و `validation` را و همچنین مقدار بایاس را صفر و وزن ها را مقدار کوچک و رندوم ست میکنیم.

- `calculate ()` : در این تابع یکی از سمپل ها را میگیریم و مقدار خروجی را محاسبه میکنیم. ابتدا وزن ها را در `w` ضرب کرده و با مقدار `bias` جمع میکنیم و بعد از `sign` گرفتن از خروجی نوروں ها و تبدیل به مقدار منفی ۱ و مثبت ۱ ، خروجی همه نوروں ها را `AND` میکنیم.

- `Find_candidate ()` : در این تابع یکی از نمونه ها را به عنوان ورودی میگیریم و به صورت حریصانه، اندیس یکی از نوروں هایی که تغییر علامت آن باعث تغییر علامت خروجی می شود را برمی گردانیم. بدلیل اینکه تابع خروجی ما `AND` است و ممکن است با تغییر علامت یک نوروں همچنان خروجی تغییر نکند و مدل کلا وزن ها را اصلاح نکند، از یک سیاست ملایم تر استفاده کردیم و نوروں هایی که علامت آن ها با علامت میانگین نوروں ها تفاوت داشت را به عنوان نوروں کاندید انتخاب کردیم و در ادامه از این نوروں های کاندید، نوروںی که مقدار کمتری داشت را به عنوان نوروں نهایی انتخاب میکنیم و اندیس آن را برای بروزرسانی وزن هایش برمی گردانیم.

- `train ()` : در این تابع، بوسیله توابع تعریف شده بالا به آموزش مدل و بروزرسانی وزن ها می پردازیم و هر بار وزن های نوروںی که تابع `find_candidate ()` انتخاب کرده را بروزرسانی میکنیم.

- `predict ()` : در این تابع داده ها را میگیریم و در وزن ضرب و با بایاس جمع میکنیم.

شبهه کد این کلاس در صفحه بعد آورده شده است:

٢ سوال دوم : Madaline and Adaline

الگوریتیم 1 Training Madaline Algorithm

Input: Training data (X_{train}, Y_{train}) , Validation data (X_{val}, Y_{val}) , learning rate η , regularization λ , number of epochs

Output: Trained weights and biases

Initialize: Set initial parameters and variables (number of neurons m , learning rate, weights, biases, etc.)

Function *CALCULATE*(x):

```
net =  $x \cdot w + bias$ 
 $z = sign(net)$ 
if  $all(z = 1)$  then
| return 1
else
| return -1
end
```

Function *FIND_CANDIDATE*(x):

```
net =  $x \cdot w + bias$ 
 $z = sign(net)$ 
output = 1 if  $mean(z) \geq 0$  else -1
if output  $\neq z$  for some neurons then
| store their indices in wrong_indices
| return index with smallest  $|net|$  among wrong_indices
else
| return None
end
```

Function *TRAIN*(num_epochs, λ):

```
for each epoch do
| for each  $(x_i, y_i)$  in training data do
| |  $y_{pred} = CALCULATE(x_i)$ 
| | if  $y_{pred} \neq y_i$  then
| | |  $c = FIND\_CANDIDATE(x_i)$ 
| | | if  $c$  is not None then
| | | | select neuron  $c$  and update its weights and bias
| | | end
| | end
| end
| Evaluate model performance on validation dataset
end
```

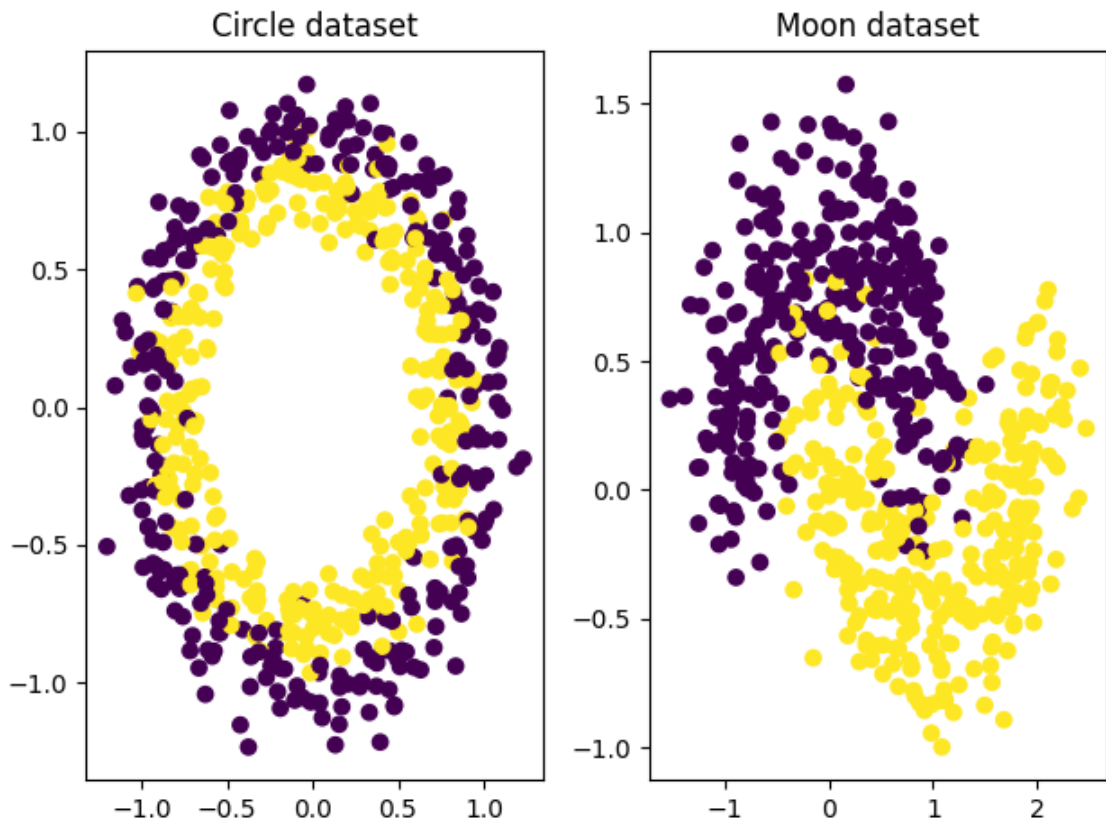
Function *PREDICT*(X):

```
for each sample  $x$  in  $X$  do
| output = CALCULATE( $x$ )
end
return all outputs
```

۲ سوال دوم : Madaline and Adaline

۲-۲-۲ داده ها و تنظیمات آموزشی

داده های Moon و Circle را طبق صورت مسئله با استفاده از کتابخانه Sklearn می خوانیم و مدل را یکبار با جریمه های L2 داده شده آموزش می دهیم و در ادامه نتایج آن را می بینیم.

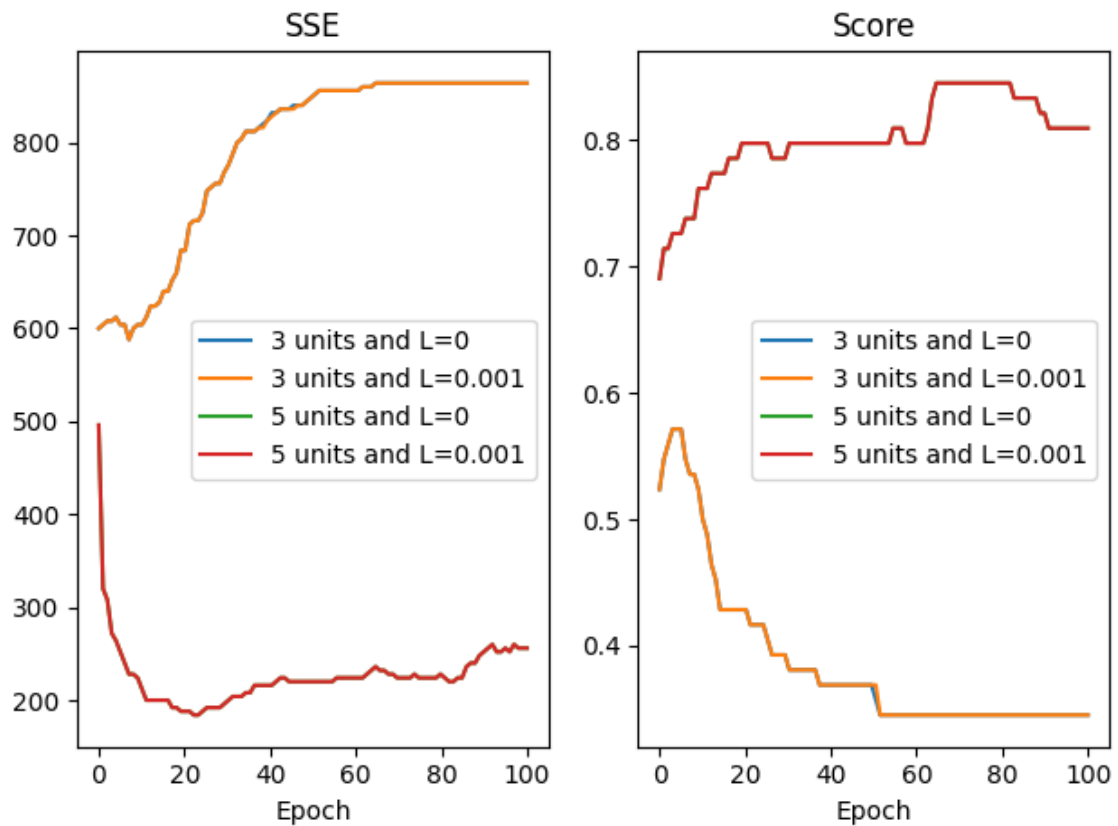


شکل ۹: Moon and Circle dataset

۲-۲-۳ ارزیابی و نمودارها

در این بخش ما برای هر داده هر دو مقدار جریمه و هر دو مقدار برای تعداد نوروں های میانی را تست می کنیم و بهترین را گزارش می کنیم . ابتدا دیتا ست Moon را بررسی می کنیم و نمودار خطا و دقت را بر حسب دوره با پارامتر های مختلف نشان می دهیم:

۲ سوال دوم : Madaline and Adaline



شکل ۱۰: SSE and Accuracy (L stand for L2 regularization)

همچنین مقدار دقت را برای هر کدام از مدل ها به صورت زیر بدست آوردیم:

جدول ۶: دقت مدل ها

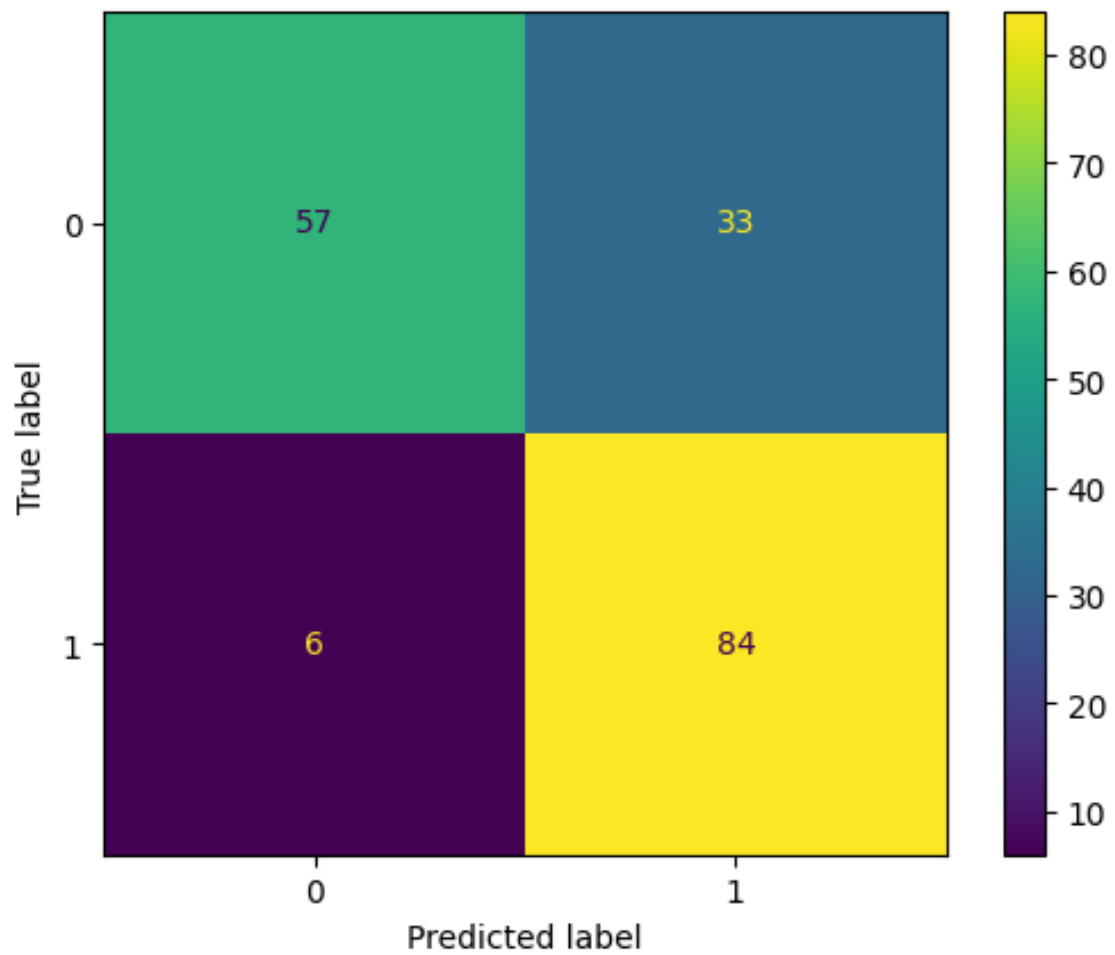
3 units with L = 0	3 units with L = 0.001	5 units with L = 0	5 units with L = 0.001
0.34	0.34	0.78	0.78

با توجه به این داده ها بهترین مدل از بین این مدل ها را مدل با ۵ نورون انتخاب میکنیم (مقدار L2 در این سید

تاثیری ندارد)

۲ سوال دوم : Madaline and Adaline

برای این مدل ماتریس درهم‌ریختگی را رسم می‌کنیم و متریک‌های مورد نظر را گزارش می‌کنیم:



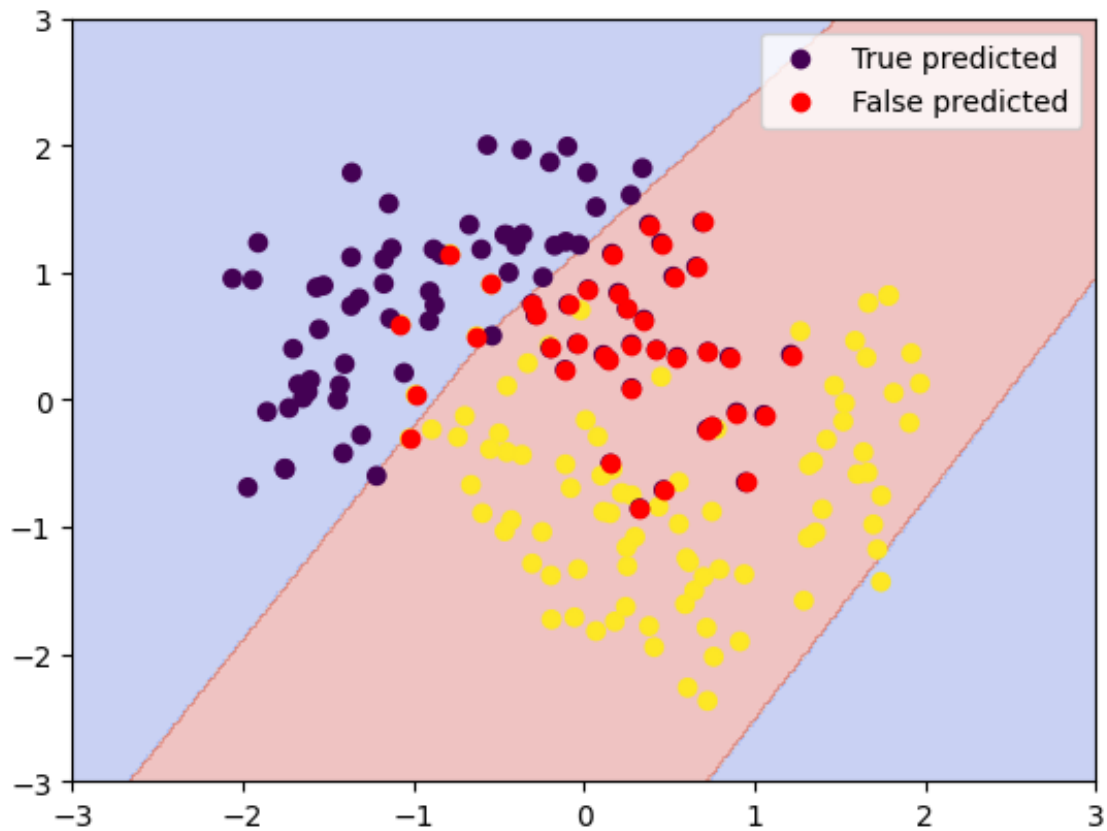
شکل ۱۱: ماتریس درهم‌ریختگی

جدول ۷: ارزیابی مدل

ROC_AUC	F1_score	Recall	Precision
0.783	0.812	0.933	0.718

۲ سوال دوم : Madaline and Adaline

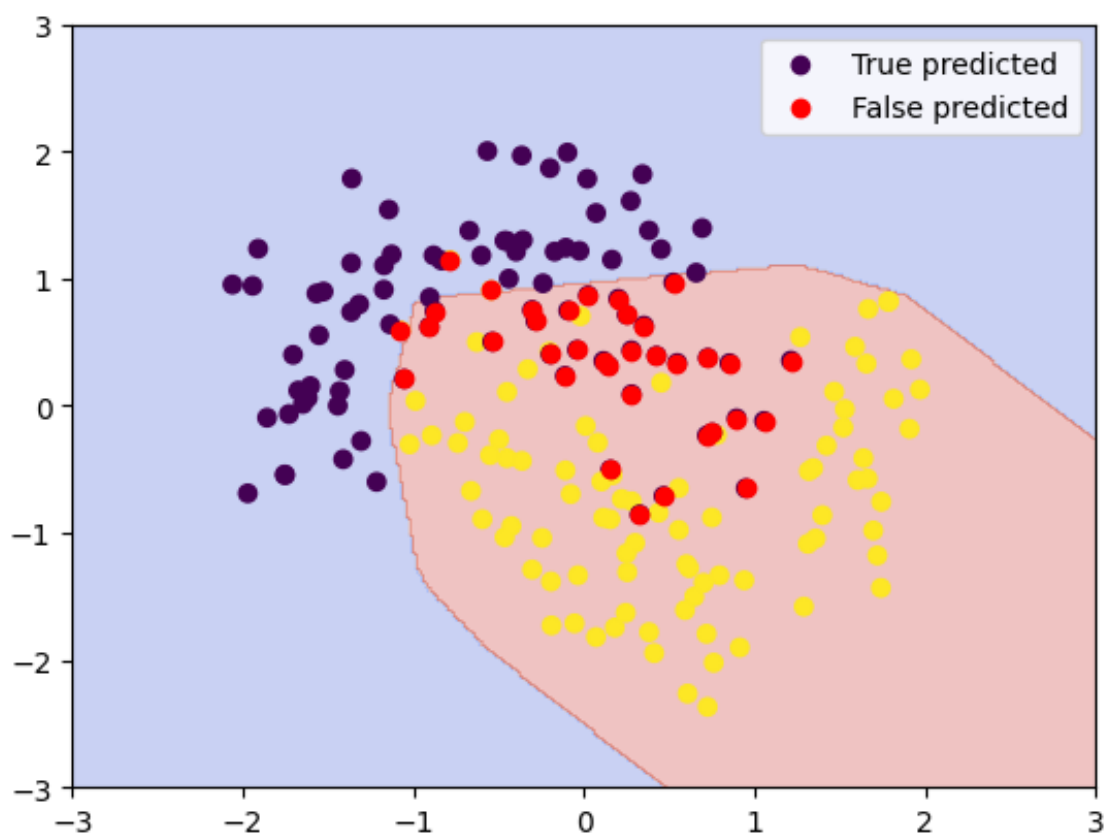
حال مرز تصمیم را برای این مدل ۵ نورونی نشان می‌دهیم و نقاط اشتباه را مشخص می‌کنیم:



شکل ۱۲: dataset Moon for boundaries decision Madaline

۲ سوال دوم : Madaline and Adaline

با توجه به شکل میدانیم که می‌توان در صورتی که تعداد نوروں های Madaline را افزایش داد این مرز تصمیم بهتر عمل کند و دقت بالاتری داشته باشد. در شکل زیر نوروں ها را به ۲۰ افزایش می‌دهیم دقت به ۸۱ درصد می‌رسد و نمودار آن به صورت زیر خواهد بود:



شکل ۱۳: Madaline decision boundries with 20 neurons

۲ سوال دوم : Madaline and Adaline

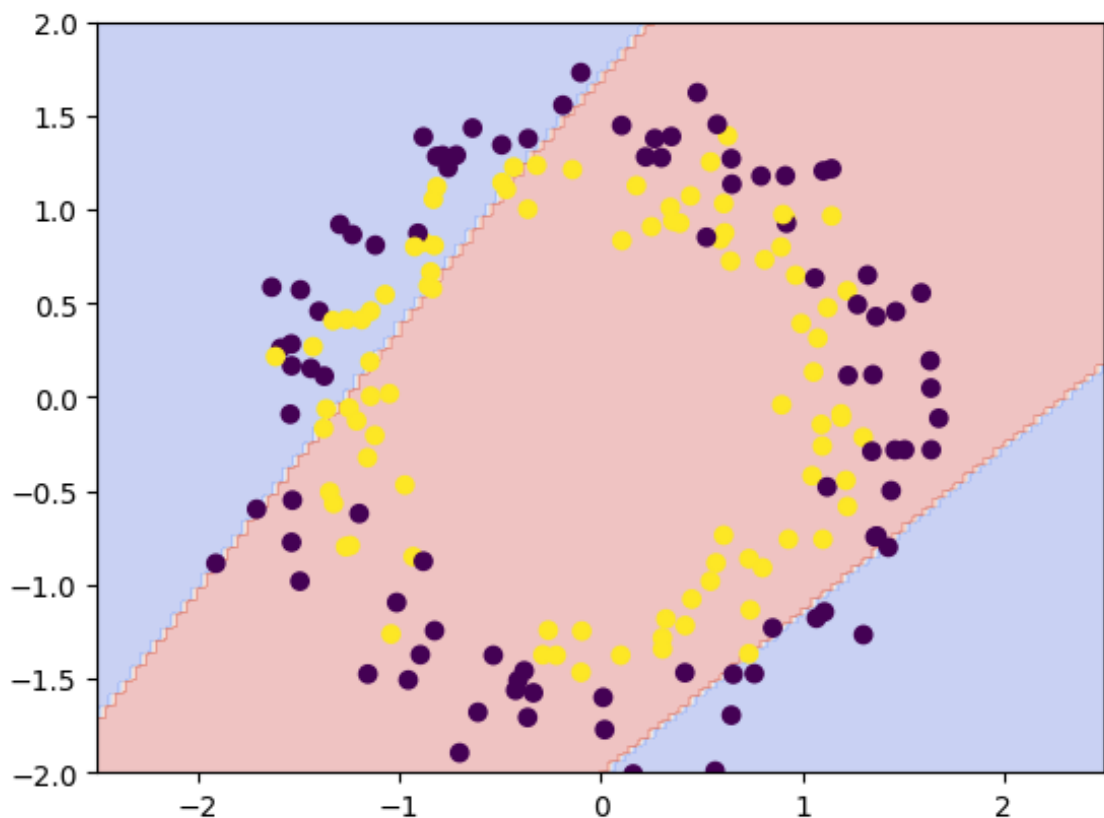
حال همین نتایج را برای دیتا ست Circle می‌آوریم. ابتدا دقت هر ۴ مدل را گزارش می‌کنیم:

جدول ۸: دقت مدل‌ها

3 units with $L = 0$	3 units with $L = 0.001$	5 units with $L = 0$	5 units with $L = 0.001$
0.51	0.51	0.57	0.57

همانطور که مشخص است با این پارامتر ها تقریباً می‌توان گفت مدل اصلاح‌یاد نگرفته و میتوان در شکل زیر

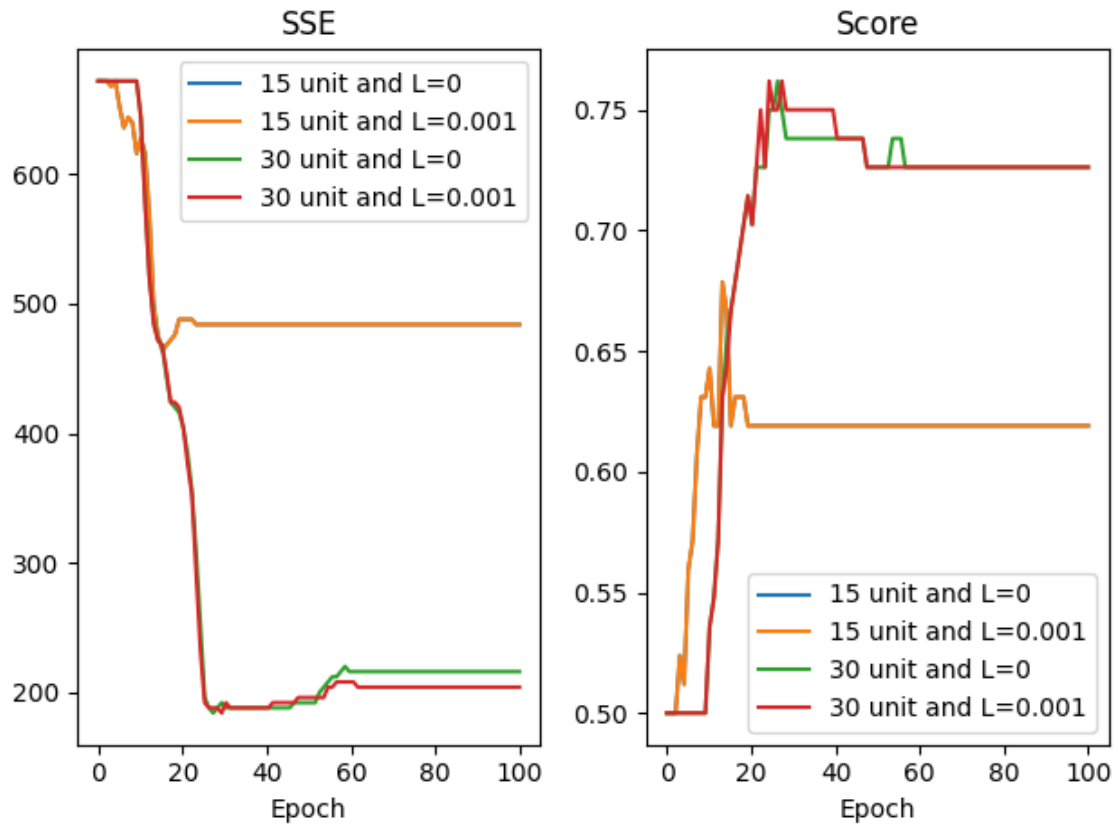
هم مشاهده کرد:



شکل ۱۴: Madaline decision boundaries for Circle dataset

می‌توان حدس زد که بعضی از خطوط روی هم افتاده اند و ۵ خط به صورت کلی برای مشخص کردن محدوده دایره مناسبی نیست و باید مقدار نوروں ها افزایش پیدا کند. به همین منظور ما مدل را بجای ۳ و ۵ نوروں روی ۱۵ و ۳۰ تا آموزش می‌دهیم و نتایج را گزارش می‌کنیم. نتایج با پارامتر های جدید به صورت زیر است:

۲ سوال دوم : Madaline and Adaline



شکل ۱۵: SSE and Accuracy during training

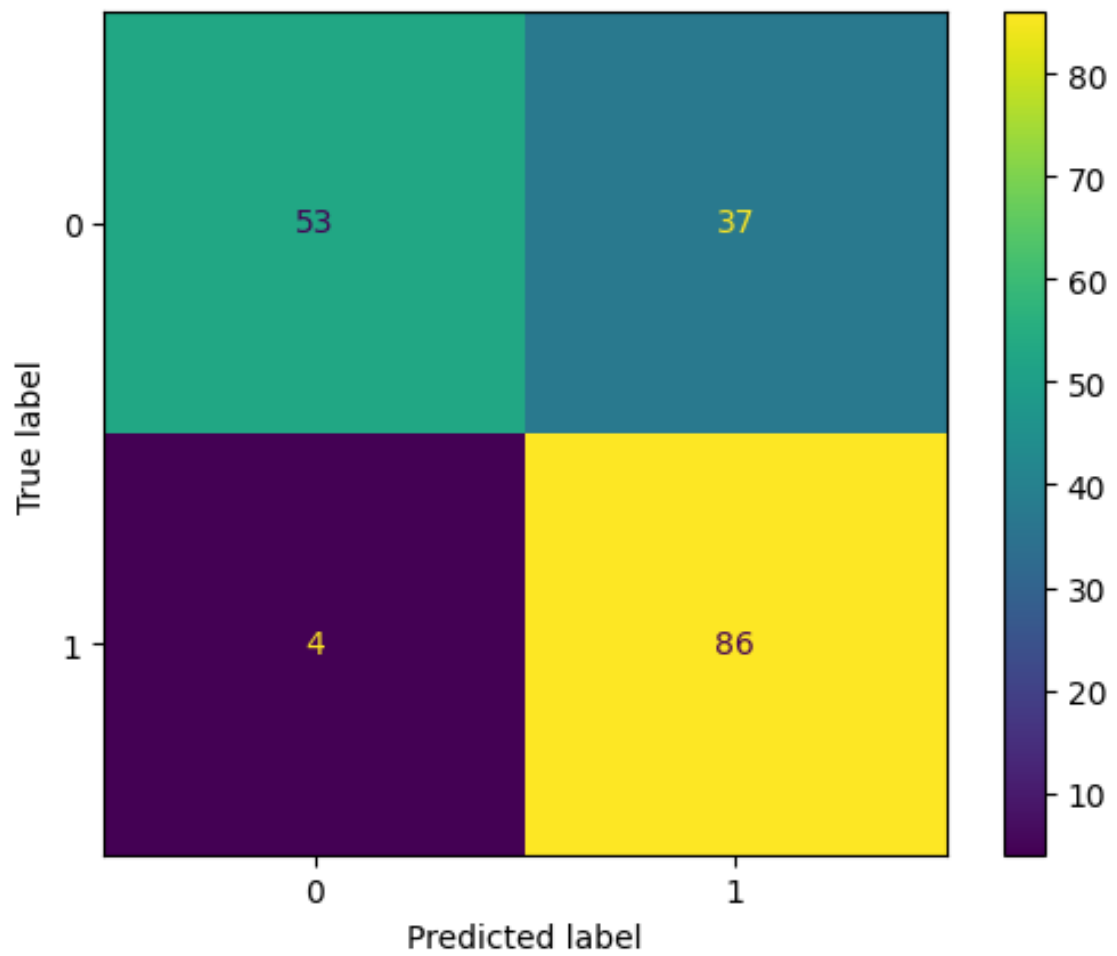
جدول ۹: دقت مدل ها

15 units with $L = 0$	15 units with $L = 0.001$	30 units with $L = 0$	30 units with $L = 0.001$
0.63	0.63	0.72	0.74

مشاهده می‌کنیم که مدل وزن ها را بدرستی بروزسانی کرده است. طبق این نتایج بهترین مدل را مدل ۳۰

نورونی و $L2 = 0.001$ انتخاب می‌کنیم و متریک های صورت مسئله را برای این مدل گزارش می‌کنیم:

۲ سوال دوم : Madaline and Adaline



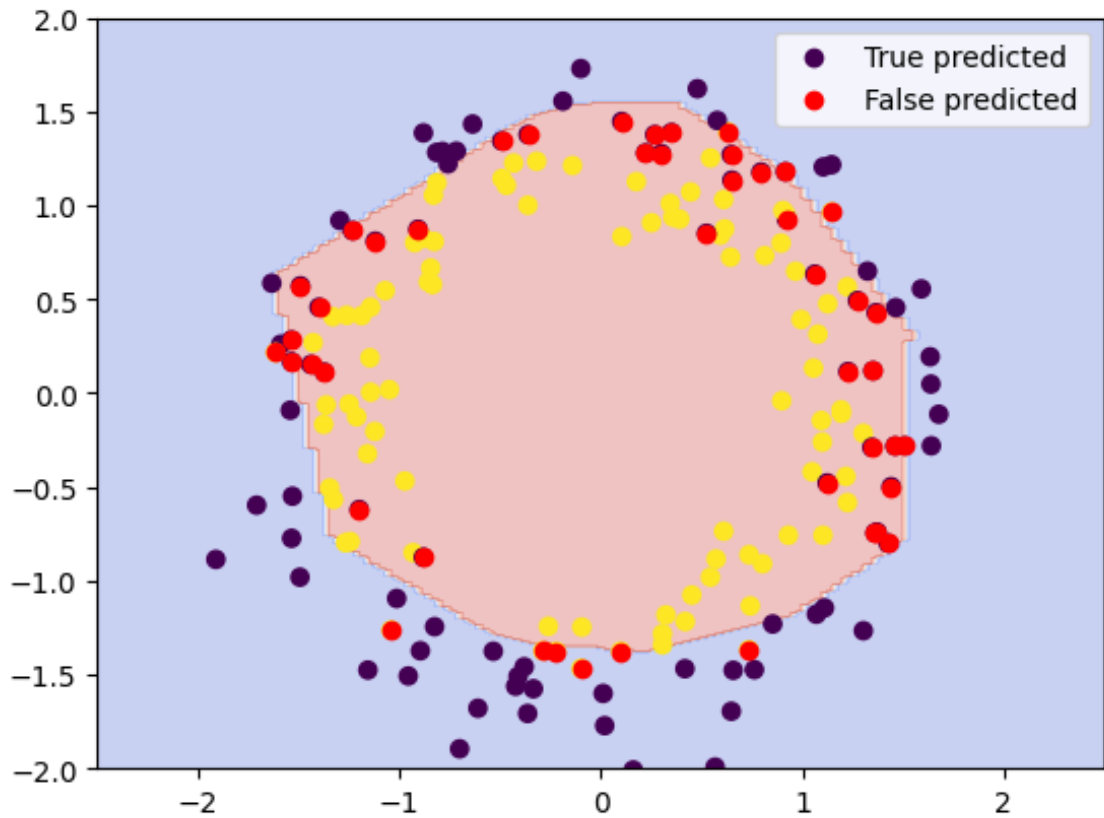
شکل ۱۶: ماتریس درهم‌ریختگی

جدول ۱۰: ارزیابی مدل

ROC_AUC	F1_score	Recall	Precision
0.772	0.808	0.956	0.699

۲ سوال دوم : Madaline and Adaline

حال مرز تصمیم مدل را رسم میکنم:



شکل ۱۷ : Madaline decision boundaries for Circle dataset

که مشاهده می‌شود این مدل از مدل ۵ نوروونی بهتر عمل می‌کند.

۲ سوال دوم : Madaline and Adaline

۴-۲-۲ مطالعه ابر پارامتر ها

نتایج داده ها به صورت خلاصه در جدول زیر آورده شده است:

جدول ۱۱: خلاصه نتایج برای Seed صفر

-	Recall	F1_score	ROC_AUC	Precision	Accuracy
Moon	0.93	0.81	0.78	0.72	0.78
Circle	0.95	0.81	0.77	0.70	0.74

جدول ۱۲: خلاصه نتایج برای Seed یک

-	Recall	F1_score	ROC_AUC	Precision	Accuracy
Moon	1	0.87	0.85	0.77	0.84
Circle	0.95	0.77	0.72	0.65	0.72

جدول ۱۳: خلاصه نتایج برای Seed دو

-	Recall	F1_score	ROC_AUC	Precision	Accuracy
Moon	1	0.78	0.72	0.64	0.75
Circle	0.97	0.82	0.78	0.71	0.78

بدلیل خطی بودن مدل Madaline در حالت کم برآزش، مدل روی داده های Moon و Circle مرز تصمیم مناسبی ایجاد نمیکرد و بسیاری از نمونه ها را اشتباه طبقه بندی میکرد. با افزایش تعداد نورون ها ، دقت مدل تا یک حد آستانه ای افزایش یافت و پس از آن ثابت ماند. زیرا نورون ها خطوط مشابهی نشان میدادند و بنابراین روی این داد ها بیش برآزش رخ نمیداد. به طور خلاصه، مدل ما با داده های غیر خطی و نورون های محدود ۳ یا ۵ تایی مخصوصا روی داده های Circle دچار کم برآزش بود که با افزایش نورون ها این مشکل تا حد خوبی برطرف می شد.

۳ سوال سوم: رگرشن با استفاده از mlp

۱-۳ دریافت دیتاست و انجام پیش پردازش های لازم

۱-۱-۳ مدیریت داده های گمشده

ابتدا تعداد داده های گمشده مربوط به هر ستون جدول را بررسی می کنیم:

```
Missing values per column BEFORE handling:
longitude          0
latitude           0
housing_median_age  0
total_rooms         0
total_bedrooms     207
population          0
households          0
median_income       0
median_house_value  0
ocean_proximity    0
dtype: int64
```

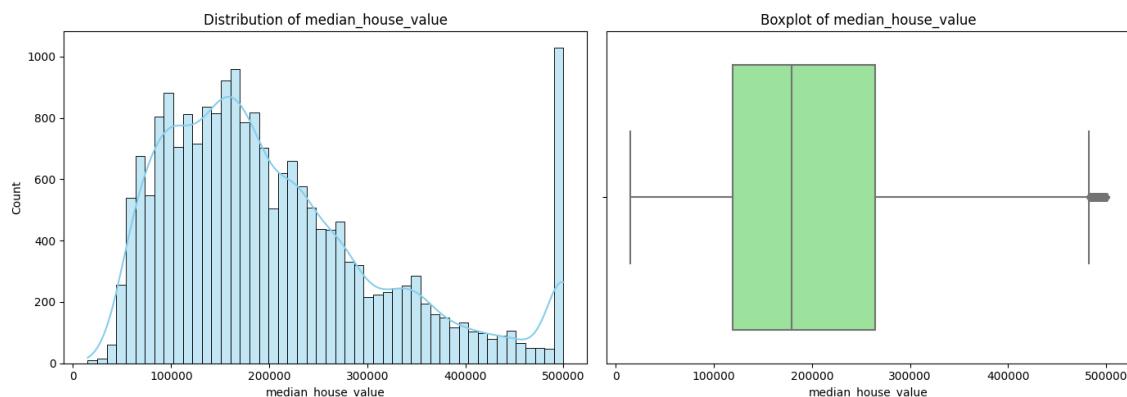
شکل ۱۸: تعداد داده های گمشده مربوط به هر ستون دیتاست

با توجه به خروجی، مشخص است که تنها ستونی که داده گمشده دارد، ستون total_bedrooms می باشد. حال می توانیم برای مدیریت داده های گمشده، ریکوردهای دارای گمشدگی را به طور کل حذف کنیم. و با توجه به تعداد بالای ریکوردها در نسبت با تعداد داده های گمشده، این امر می تواند منطقی و معقول باشد. اما در اینجا ترجیح دادیم که از روش های جاگذاری یا imputation استفاده کنیم. همچنین با توجه به اینکه می دانیم جاگذاری با میانه یعنی median imputation در مقابل داده های پرت robust است، بنابراین از جاگذاری با میانه استفاده می کنیم.

۲-۱-۳ حذف داده های پرت از قسمت ستون هدف

ابتدا توزیع ستون هدف را رسم می کنیم تا برآوردی از شکل آن و در نتیجه استراتژی انتخابی برای حذف داده های پرت داشته باشیم.

۳ سوال سوم: رگرشن با استفاده از mlp



شکل ۱۹: نمودار جعبه‌ای و نمودار هیستوگرام داده‌های ستون هدف

همان طور که مشخص است، در اینجا یک چولگی مثبت مشاهده می‌شود. این مسئله نشان می‌دهد که تخمین نرمال بودن برای توزیع ستون هدف نامعقول است و بنابراین روش IQR را به روش Z-Score، در میان روش‌های ساده، ترجیح می‌دهیم.

حال روش IQR را برای حذف داده‌های پرت اعمال می‌کنیم. دقت شود که برای محاسبه Q_1 از $quantile(0,25)$ و برای محاسبه Q_3 از $quantile(0,75)$ استفاده می‌کنیم.

```
Number of records BEFORE removing outliers: 20640
Number of records AFTER removing outliers: 19569
```

شکل ۲۰: نمایش تعداد ریکوردها قبل و بعد از حذف داده‌های پرت

همان طور که مشخص است قبل از حذف داده‌های پرت، تعداد ریکوردها ۲۰۶۴۰ بوده است که بعد از حذف تعداد، به ۱۹۵۶۹ رسیده است. یعنی تعداد ۱,۰۷۱ ریکورد به عنوان داده پرت شناسایی شده‌اند که به درستی حذف شدند.

۳-۱-۳ تبدیل فیچرهای categorical به یک مقدار قابل قبول

ابتدا ستون‌های categorical را شناسایی کرده و سپس با استفاده از تابع `pd.get_dummies` کدگذاری one-hot را اعمال می‌کنیم. همان طور که مشخص است تنها ستون دسته‌ای یا کتگوریکال، ستون `ocean_proximity`

می‌باشد.

۲-۳ پیاده‌سازی یک شبکه عصبی ساده

در این قسمت ابتدا دیتاست را به سه بخش آموزش، ارزیابی و تست، با میزان درصد مطلوب برای هر یک تقسیم می‌کنیم. تنها دقت شود که ابتدا ۰,۱۰ از داده‌ها را به‌عنوان داده‌های تست کنار گذاشته‌ایم. سپس از ۰,۹۰ باقی‌مانده باید ۸ سهم از آن را به‌عنوان داده آموزش جدا کنیم و یک سهم آن را به‌عنوان داده ارزیابی کنار بگذاریم و این امر در صورتی امکان‌پذیر است که ۰,۱۱۱۱ از داده‌ها را به‌عنوان داده‌های ارزیابی جدا کرده و باقی‌مانده را داده آموزش در نظر بگیریم.

سپس معماری خواسته‌شده را با استفاده از کتابخانه tensorflow پیاده‌سازی کرده‌ایم.

Layer (type)	Output Shape	Param #
dense_10 (Dense)	(None, 8)	104
re_lu_3 (ReLU)	(None, 8)	0
dense_11 (Dense)	(None, 1)	9

Total params: 113 (452.00 B)

Trainable params: 113 (452.00 B)

Non-trainable params: 0 (0.00 B)

شکل ۲۱: نمایش ساختار و تعداد پارامترهای شبکه عصبی ساده پیشنهادی

همان‌طور که مشخص است، تعداد ۱۱۳ پارامتر برای این ساختار وجود دارد که همه آنها باید با استفاده از داده‌های آموزش، ترین شوند.

۳-۳ آموزش شبکه عصبی ساده

در این قسمت ابتدا به کمک کلاس RegressionMetrics ، برای هر متریک خواسته شده، یک متود تعریف کرده ایم که معیار مطلوب را محاسبه کرده و برمیگرداند. حال با توجه به خواسته سوال، به معرفی هر کدام از معیارها می پردازیم.

- **میانگین مربعات خطا (MSE):** میانگین مربعات خطا یا Mean Squared Error به صورت زیر تعریف می شود:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

این معیار مقدار میانگین توان دوم اختلاف بین مقادیر واقعی و پیش بینی شده را اندازه گیری می کند. هر چه مقدار MSE کمتر باشد، عملکرد مدل بهتر است. این معیار نسبت به داده های پرت حساس است.

- **ریشه ی میانگین مربعات خطا (RMSE):** ریشه ی میانگین مربعات خطا یا Root Mean Squared Error برابر است با:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} = \sqrt{MSE}$$

این معیار تفسیرپذیری بهتری نسبت به MSE دارد، زیرا واحد آن با واحد متغیر هدف یکسان است. از RMSE برای مقایسه ی خطاها در مقیاس واقعی داده استفاده می شود.

- **میانگین قدر مطلق خطا (MAE):** میانگین قدر مطلق خطا یا Mean Absolute Error به صورت زیر تعریف می شود:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

این معیار میانگین قدر مطلق اختلاف بین مقادیر واقعی و پیش بینی شده را نشان می دهد. برخلاف MSE، نسبت به داده های پرت مقاوم تر است و به طور مستقیم نشان دهنده ی میانگین خطای مدل در واحد اندازه گیری داده ها است.

- **ضریب تعیین (R^2):** ضریب تعیین یا Coefficient of Determination میزان برازش مدل را بیان

۳ سوال سوم: رگرشن با استفاده از mlp

می‌کند و به شکل زیر محاسبه می‌شود:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

مقدار R^2 بین ۰ و ۱ قرار دارد (و در برخی موارد می‌تواند منفی شود). هرچه مقدار آن به ۱ نزدیک‌تر باشد، مدل توانسته است واریانس داده‌ها را بهتر توضیح دهد.

حال کاربرد هر یک از معیارها را بررسی می‌کنیم:

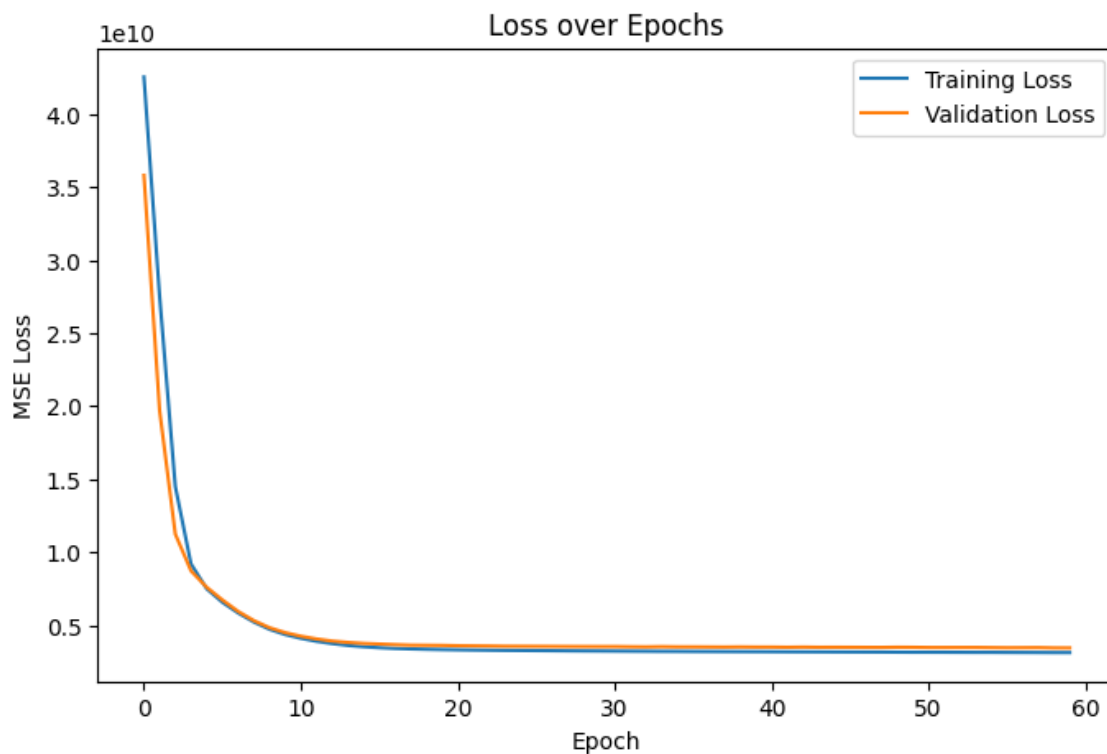
- **MSE**: معمولاً برای آموزش مدل‌های رگرسیونی و شبکه‌های عصبی به‌منظور کمینه‌سازی خطاهای بزرگ استفاده می‌شود.
- **RMSE**: برای ارزیابی دقت مدل در مقیاس واقعی داده‌ها، به‌ویژه در مسائل پیش‌بینی و سری‌های زمانی کاربرد دارد.
- **MAE**: در داده‌های دارای مقادیر پرت یا زمانی‌که تفسیر ساده‌ی خطا اهمیت دارد، مورد استفاده قرار می‌گیرد.
- R^2 : برای سنجش میزان برازش مدل و مقایسه‌ی عملکرد مدل‌های رگرسیونی مختلف به‌کار می‌رود.

حال پیش از آموزش مدل روی مجموعه داده آموزش، ستون‌های عددی را از ستون‌های one-hot شده استخراج کرده و به‌کمک تابع `StandardScaler()` از کتابخانه `sklearn`، یک نرمال‌سازی استاندارد روی آنها انجام می‌دهیم. دقت شود که در این فرآیند نرمالیزیشن، از شخصیت مجموعه داده‌ترین استفاده می‌شود تا یک نرمال‌سازی بر روی مجموعه داده‌ترین (آموزش)، ارزیابی و تست اعمال شود. زیرا اگر برای نرمال‌سازی داده‌های تست و ارزیابی از شخصیت و ویژگی‌های خود این مجموعه داده‌ها استفاده شود، به‌اشتباه درگیر `Data Leakage` می‌شویم.

سپس در ۶۰ اپیک، با $batch_size = 64$ طبق خواسته سوال، آموزش را اتمام می‌دهیم.

جدول ۱۴: مقایسه‌ی معیارهای ارزیابی مدل برای مدل شبکه عصبی ساده

R^2	MAE	RMSE	MSE	Dataset
0.6579	40364.4802	55799.9301	3113632201.2904	Train
0.6155	40931.3304	58629.1604	3437378450.7101	Validation
0.6733	40418.3120	55178.8465	3044705104.1831	Test



شکل ۲۲: نمودار loss برای مجموعه داده های آموزش و اعتبارسنجی در فاز آموزش شبکه عصبی ساده

طبق نمودار loss مشخص است که آموزش به درستی انجام شده است و مدل یادگیری را تا در سطح خوبی فرا گرفته است. همچنین جدول مربوطه نیز به ما نشان می دهد که خواسته سوال ارضا شده است و برای معیار R^2 مقادیری بیش از ۶۰٪ بدست آمده است.

۳-۴ طراحی و آموزش شبکه عصبی پیچیده تر

معماری پیچیده تر انتخاب شده، در شکل زیر نمایش داده شده است.

۳ سوال سوم: رگرشن با استفاده از mlp

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 128)	1,664
batch_normalization_6 (BatchNormalization)	(None, 128)	512
dropout_4 (Dropout)	(None, 128)	0
dense_17 (Dense)	(None, 64)	8,256
batch_normalization_7 (BatchNormalization)	(None, 64)	256
dropout_5 (Dropout)	(None, 64)	0
dense_18 (Dense)	(None, 32)	2,080
batch_normalization_8 (BatchNormalization)	(None, 32)	128
dense_19 (Dense)	(None, 1)	33

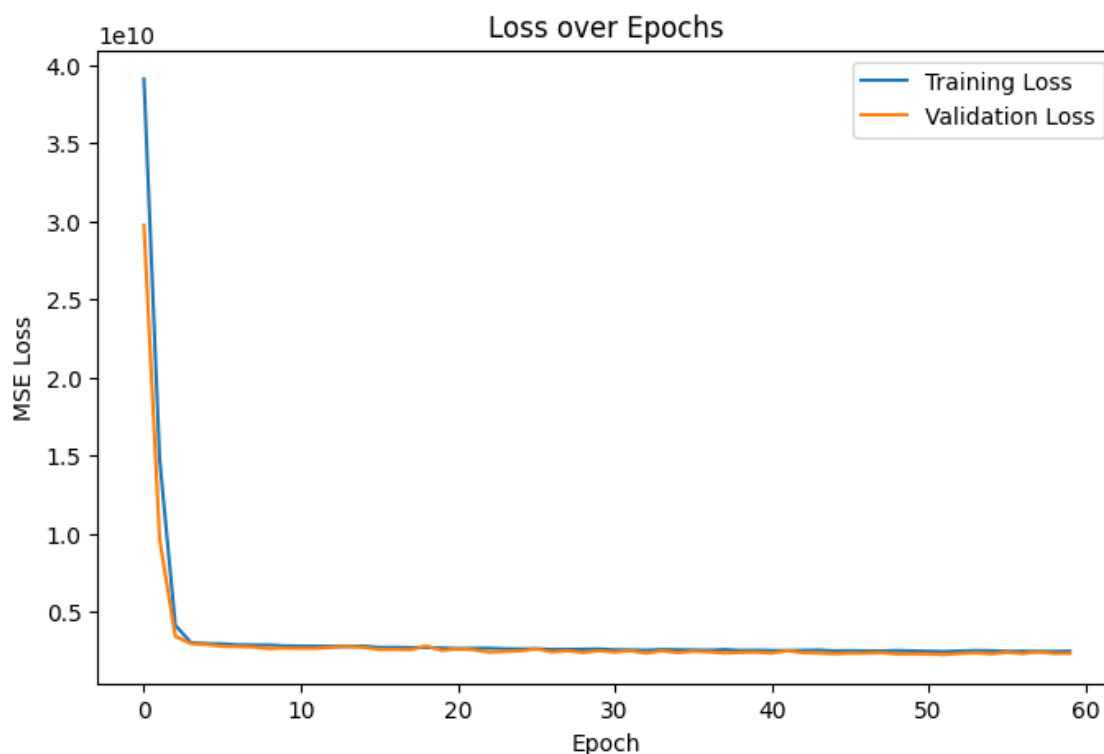
Total params: 12,929 (50.50 KB)

Trainable params: 12,481 (48.75 KB)

Non-trainable params: 448 (1.75 KB)

شکل ۲۳: نمایش ساختار و تعداد پارامترهای شبکه عصبی پیچیده‌تر انتخاب‌شده

همان‌طور که مشخص است ۱۲۹۲۹ پارامتر برای این مدل وجود دارد که تعداد ۱۲۴۸۱ تا از آنها توسط داده‌های آموزش‌ترین می‌شوند. تفاوت در اینجا این است که علاوه بر افزودن بر عمق شبکه عصبی، از Dropout و BatchNormalization نیز استفاده کرده‌ایم تا به قابلیت generalization مدل در مقابل داده‌های دیدن‌نشده بیفزاییم. حال عملکرد مدل پیچیده‌تر را بررسی می‌کنیم.



شکل ۲۴: نمودار loss برای مجموعه داده های آموزش و اعتبارسنجی در فاز آموزش شبکه عصبی پیچیده تر

جدول ۱۵: مقایسه ی معیارهای ارزیابی مدل برای مدل شبکه عصبی پیچیده تر

R^2	MAE	RMSE	MSE	Dataset
0.7749	31380.7404	45267.6831	2049163130.7108	Train
0.7399	32701.4435	48221.4069	2325304085.8537	Validation
0.7675	32356.9023	46552.2196	2167109152.4674	Test

طبق نمودار loss مشخص است که آموزش به درستی انجام شده است و مدل یادگیری را تا در سطح خوبی فرا گرفته است. همچنین جدول مربوطه نیز به ما نشان می دهد که خواسته سوال ارضا شده است و برای معیار R^2 مقادیری بیش از ۷۰٪ بدست آمده است.

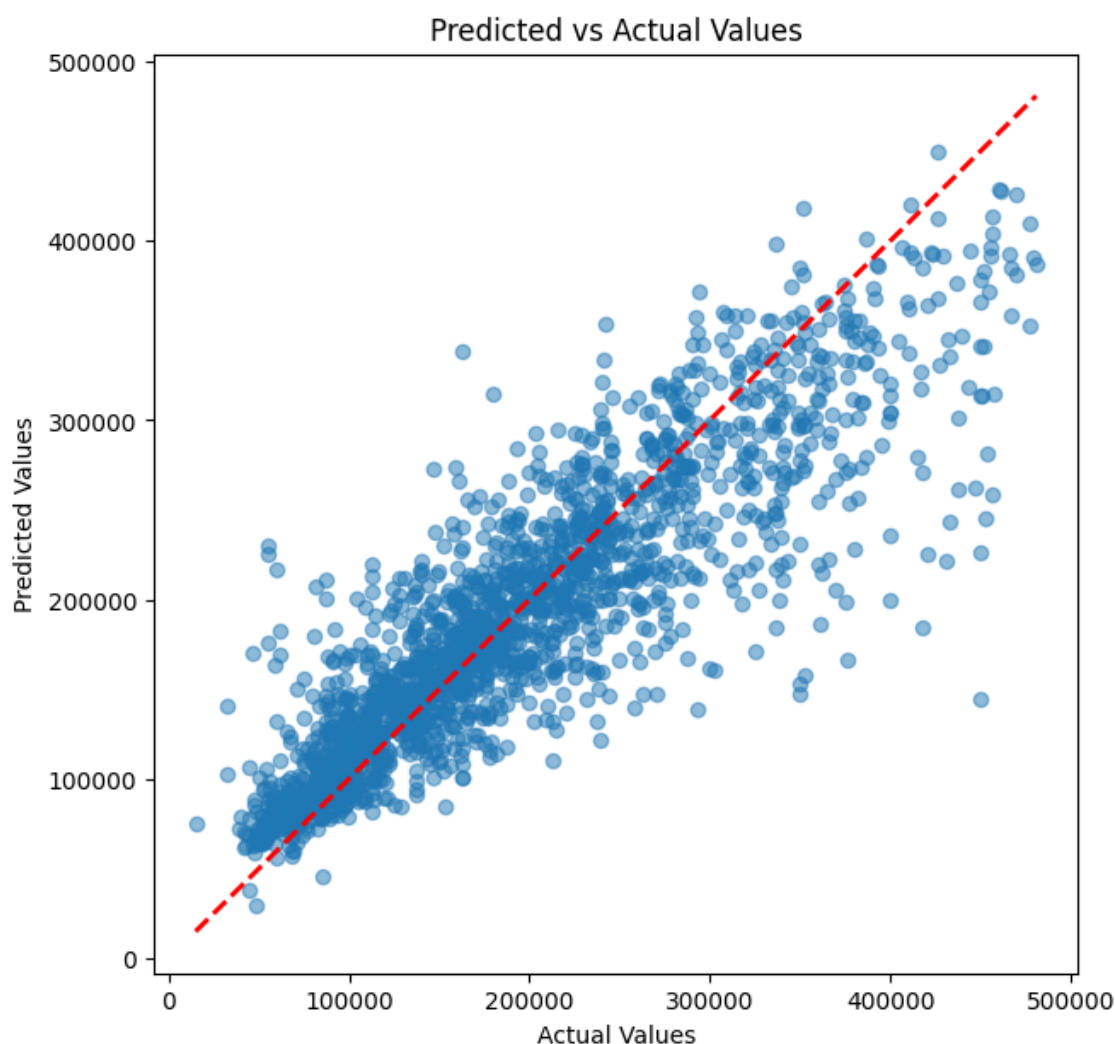
۳-۵ مقایسه

با توجه به شکل ۲۲ و شکل ۲۴ که نمودارهای loss برای دو مدل را نشان می دهند، مشخص است که برای مدل پیچیده تر، یعنی مدل دوم، خطا با شیب (تغییرات در طول زمان) بیشتری کاهش یافته است که این نشان دهنده

۳ سوال سوم: رگرشن با استفاده از mlp

قدرت بالاتر مدل است. همچنین برای مدل پیچیده‌تر، نمودار در مقادیر پایین‌تری همگرا شده است که این امر نیز نشان از قدرت یادگیری بالاتر مدل دارد. البته استفاده از BatchNormalization نیز سرعت مدل را در فاز ترین، افزایش داده است.

همچنین مقایسه معیارهای محاسبه‌شده برای مدل‌ها در جدول ۱۴ و ۱۵ نشان می‌دهد که میزان خطا به‌طور چشمگیری کاهش یافته است و انطباق یا فیت بهتری از لحاظ رگرسیونی با توجه به ضریب R^2 انجام شده است. بنابراین ساختار پیچیده‌تر پیشنهادی، به توانایی مدل برای درک الگوهای ساختاری داده افزوده است. همچنین در قسمت زیر نمودار Actual vs Predicted نمایش داده شده است.



شکل ۲۵: نمودار Actual vs Predicted برای شبکه عصبی پیچیده‌تر

۳ سوال سوم: رگرشن با استفاده از mlp

می‌دانیم که به میزانی که نقاط مشخص شده در این نمودار، به خط ۴۵ درجه نزدیکتر باشند، مدل عملکرد بهتری از خودش نشان داده است. همان طور که از شکل ۲۵ پیداست، مدل در حالتی که اندازه هدف، کوچکتر بوده است، عملکرد بهتری از خود نشان داده است و نمودار به این خط نزدیکتر بوده است. برای مقادیر بزرگتر گویا مدل عملکرد ضعیفتری از خود نشان داده است و این موجب دوری از خط ۴۵ درجه شده است. علت آن این است که همان طور که در شکل ۱۹ مشخص است، چون بیشتر داده‌ها در رنج‌های پایین‌تر مقداری قرار گرفته‌اند، مدل بیشتر در آن بازه آموزش می‌بیند و چون تعداد داده کمتری در رنج‌های بالاتر مقداری قرار گرفته‌اند، مدل از پیش‌بینی درست داده‌های تست مربوط به آنها باز می‌ماند. اما در حالت کلی، مدل عملکرد قابل قبولی از خود نشان داده است، ضمن اینکه می‌توانیم برای رسیدن به دقت‌های بالاتر، مدل‌های پیچیده‌تری را نیز استخدام کنیم.

در این بخش، سؤال اصلی مطرح می‌شود و نکات مهم مرتبط با آن شرح داده می‌شوند. دانشجو باید با توجه به دستورالعمل‌ها و توضیحات ارائه‌شده، پاسخ مناسب را ارائه کند. این قسمت شامل مقدمه‌ای کوتاه درباره موضوع، محدودیت‌ها، و اهداف مورد انتظار است تا چارچوب حل سؤال مشخص شود.

۴ طراحی شبکه Auto Encoder

۴-۱ دریافت دیتا ست و انجام پیش پردازش های لازم

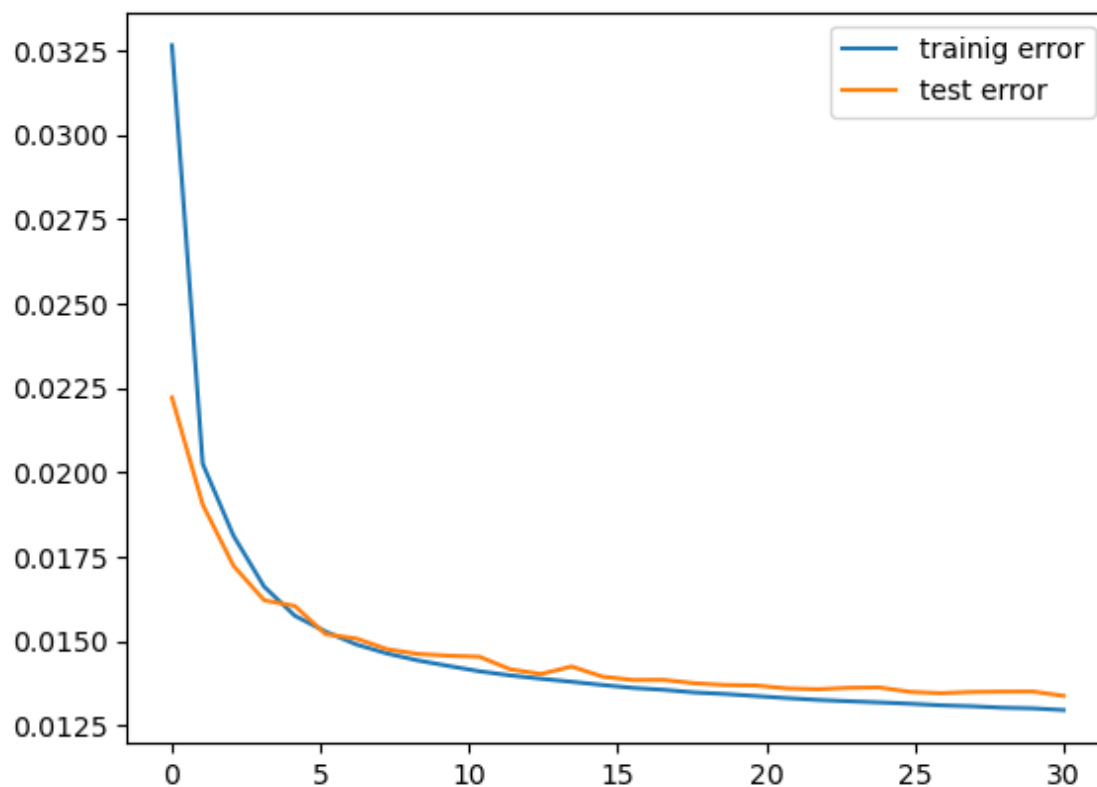
در ابتدا با کتابخانه torchvision دیتا های آموزش و تست را دانلود میکنیم و به وسیله تابع transform در همین کتابخانه داده ها را به صورت استاندارد نرمالایز میکنیم و آن ها را به بچ های ۶۴ تایی تقسیم میکنیم. در صورتی که داده ها نرمالایز نشوند ممکن است مدل پایدار نشود و وزن های مدل اعداد خیلی بالایی بگیرند. همچنین الگوریتم های بهینه سازی معمولا روی داده ها با مقیاس برابر میتوانند به درستی عمل کنند و اگر داده ها نرمال نشوند ممکن است این الگوریتم یا کار نکنند و یا کند همگرا شوند. یکی دیگر از دلایل نرمال سازی توابع tanh و sigmoid است که اگر اعداد خیلی بزرگ باشند ممکن است این توابع را وارد ناحیه اشباع کنند و گرادین آن ها در آن ناحیه صفر خواهد شد و کارایی خود را از دست میدهند

۴-۲ پیاده سازی

شبکه های autoencoder ها برای کاهش نویز و یا کاهش ابعاد مسئله کاربرد دارند و برای آموزش این شبکه ها معمولا از یک encoder چسبیده به یک decoder استفاده میشود و داده ها از سمت encoder وارد میشوند و از سمت decoder خارج میشوند و شبکه یاد میگیرد که داده های اصلی را به عنوان خروجی تولید کند و یا اگر برای حذف نویز شبکه آموزش دیده است داده های اصلی بدون نویز را به عنوان خروجی برگرداند. ما کلاس AutoEncoder را طبق موارد خواسته شده در صورت سوال تعریف کردیم و داده ها را در Encoder با سه لایه Fully Connected از ۷۸۴ بعد به ۱۰ بعد رسانیدیم و در Decoder هم به صورت قرینه آن عمل کردیم داده ها را در ۳ مرحله به ۷۸۴ بعد رساندیم.

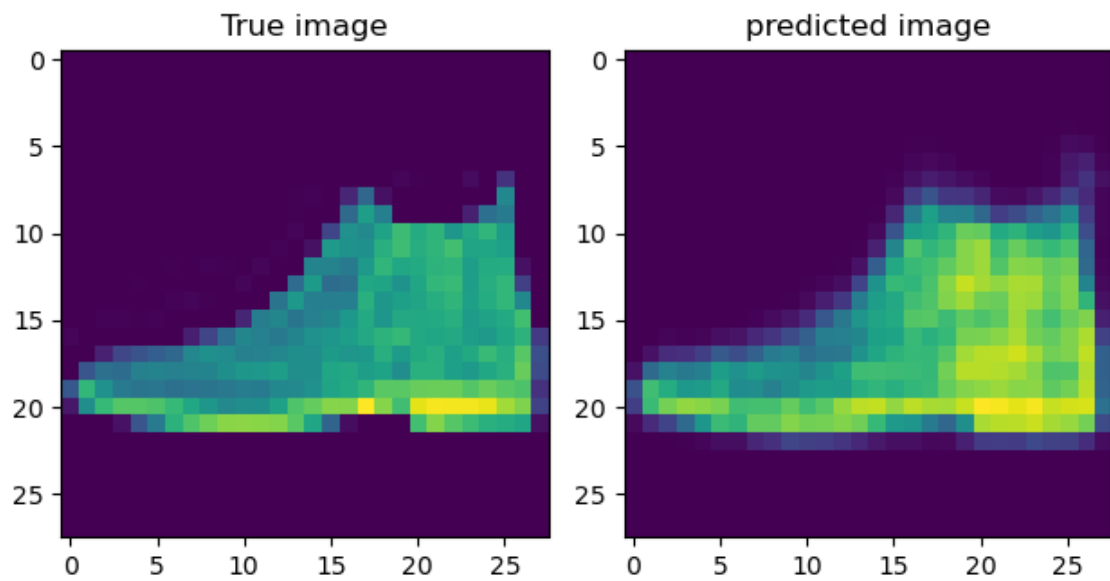
۳-۴ آموزش شبکه

در ادامه نمودار خطا بر حسب دوره را برای داده های آموزش و تست مشاهده میکنیم:

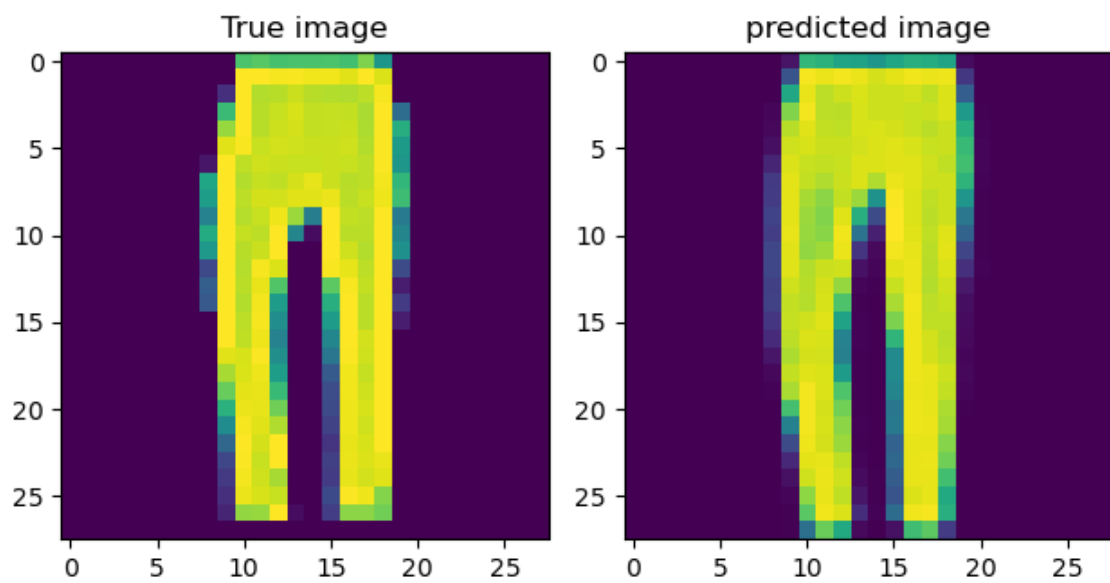


شکل ۲۶: MSE over Epochs

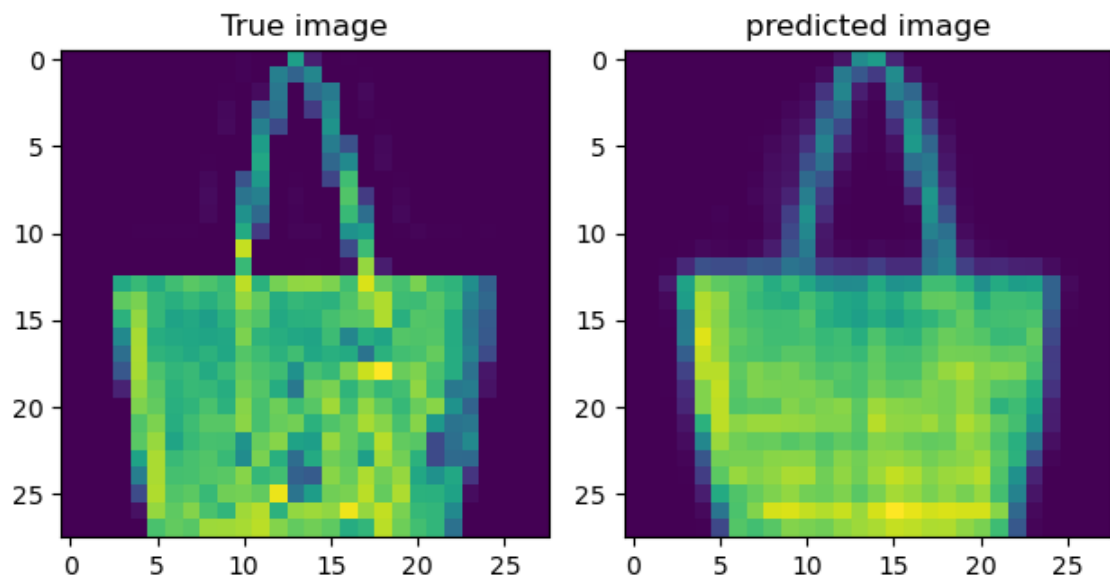
همچنین در زیر ۵ نمونه از ۵ کلاس مختلف از مجموعه تست همراه با خروجی مدل آورده شده است:



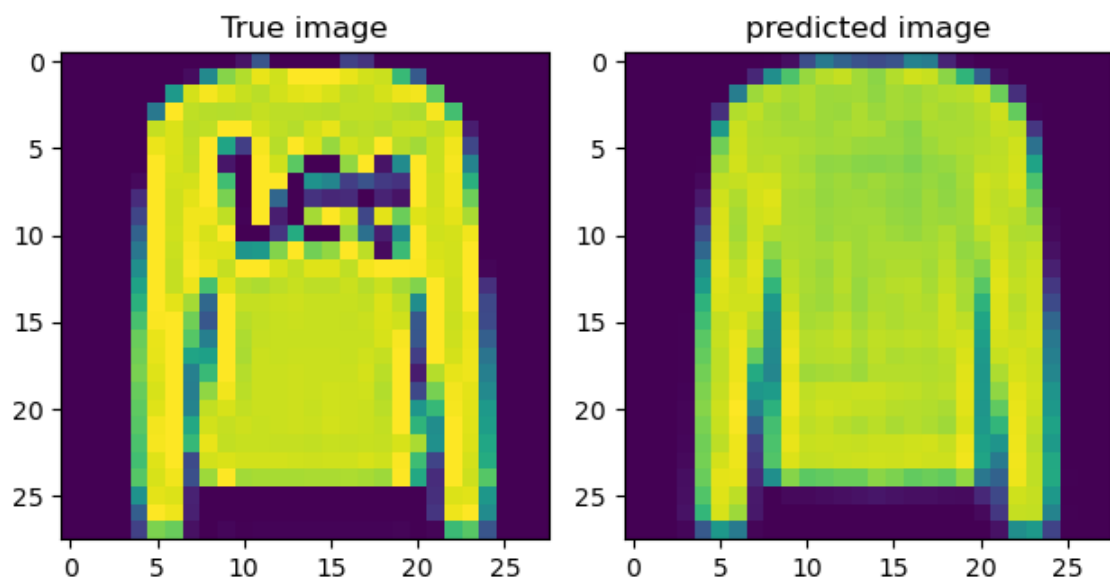
شکل ۲۷ : Ankle Boot class



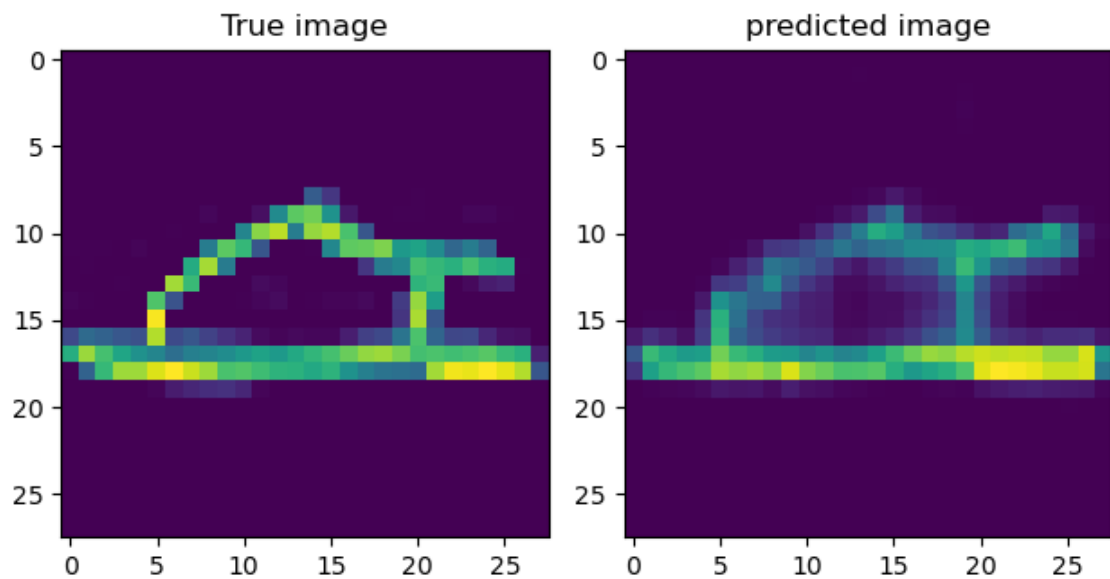
شکل ۲۸ : Trouser class



شکل ۲۹ : Bag class



شکل ۳۰ : Pullover class



شکل ۳۱: Sandal class

۴-۴ ارزیابی شبکه

۱-۴-۴ انکود کردن داده ها

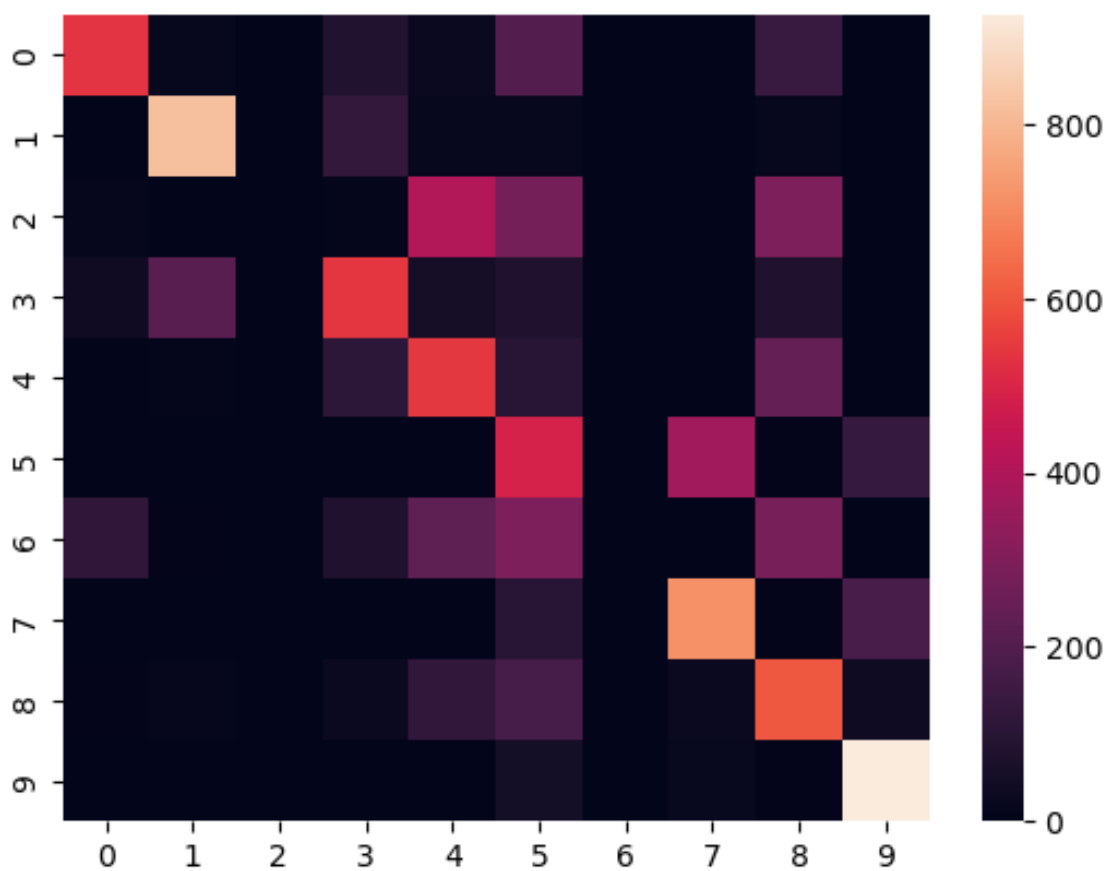
در کلاس AutoEncoder به کمک تابع Encode میتوانیم داده ها را به ابعاد کاهش یافته منتقل کنیم.

۲-۴-۴ کلاسترینگ با K-means

با کمک کتابخانه Sklearn و تابع KMeans داده ها را به ۱۰ کلاس طبقه بندی کردیم.

۳-۴-۴ تحلیل خوشه ها

برای بررسی خوشه ها ابتدا ماتریس درهم ریختگی آن را رسم می کنیم:



شکل ۳۲: ماتریس درهم‌ریختگی

همچنین کلاس بندی داده های Fashion Mnist به صورت زیر است:

جدول ۱۶: کلاس‌های دیتاست Fashion-MNIST

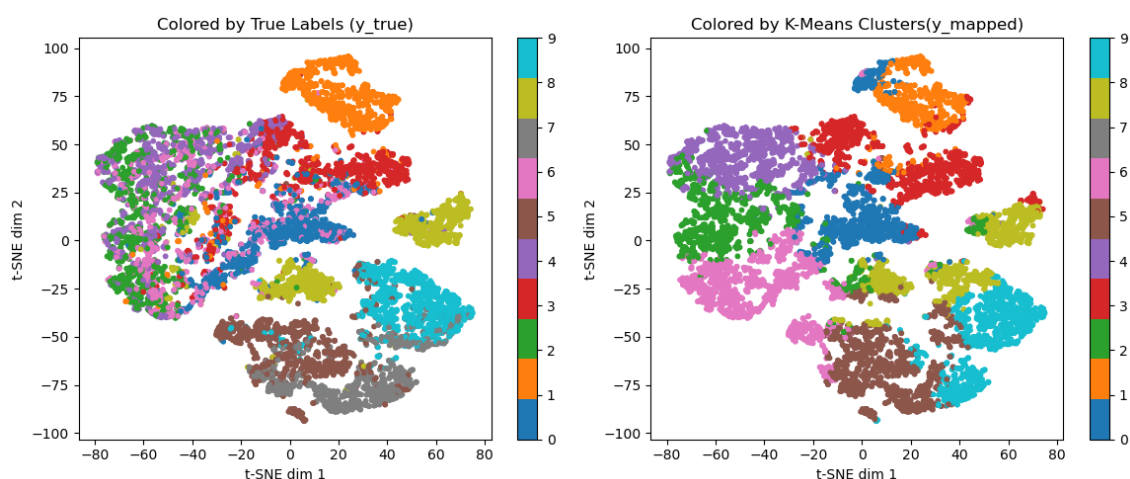
کلاس	برچسب
۰	T-shirt / Top
۱	Trouser
۲	Pullover
۳	Dress
۴	Coat
۵	Sandal
۶	Shirt
۷	Sneaker
۸	Bag
۹	Ankle boot

۴ طراحی شبکه Auto Encoder

همینطور که قابل مشاهده است روی قطر اصلی تجمع داده ها زیاد است. همچنین مشخص است که کلاس بوت ها و شلوار ها به خوبی خوشه بندی شده است ولی کلاس shirt و pullover به خوبی خوشه بندی نشده اند.

۴-۴-۴ کاهش ابعاد و t-SNE

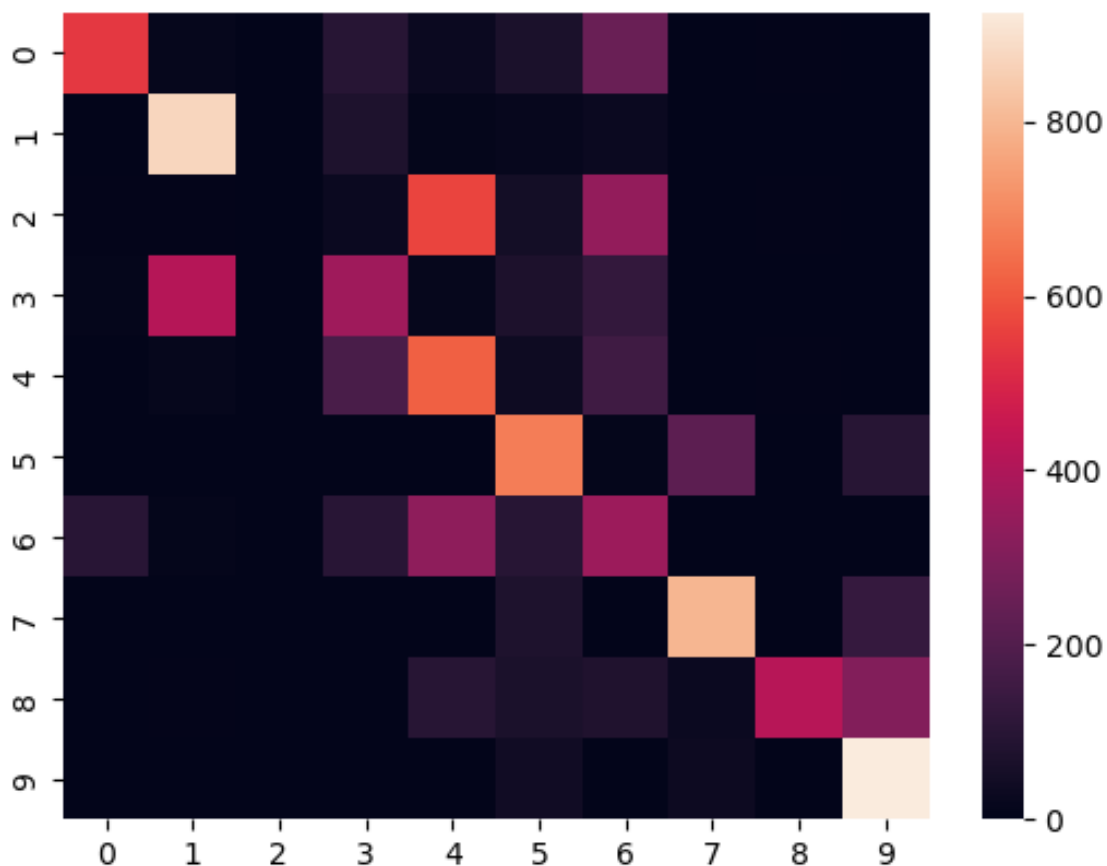
داده های تست انکود شده را با t-sne به فضای دو بعدی میبریم و به دو روش رنگ بندی میکنیم:



شکل ۳۳: 2D test dataset

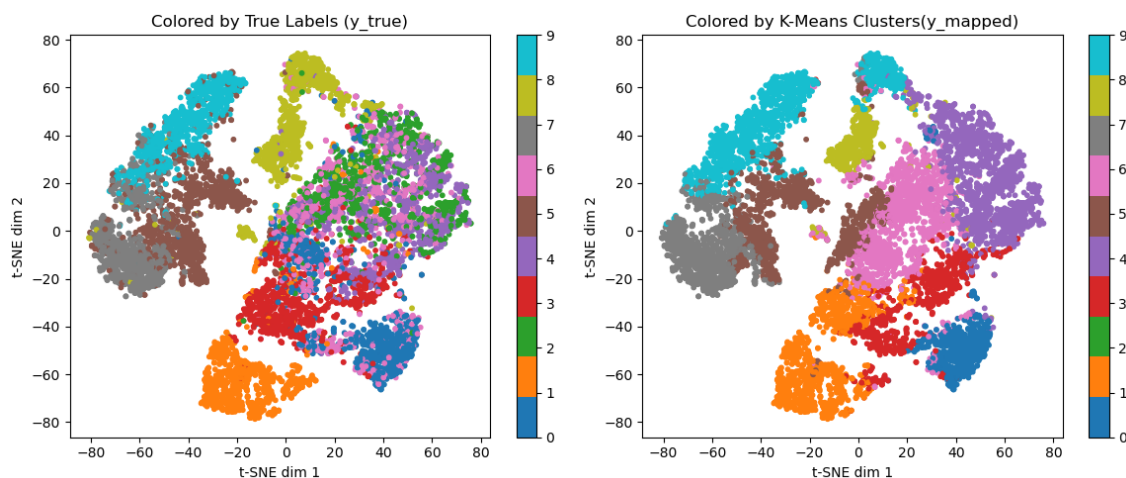
۴-۴-۵ مقایسه با داده های خام

حال همین فرایند ها را روی داده های خام انجام میدهیم. ابتدا داده ها را با K-means خوشه بندی می‌کنیم و ماتریس در هم ریختگی را رسم می‌کنیم:



شکل ۳۴: ماتریس درهم‌ریختگی

همانطور که مشخص است ماتریس درهم‌ریختگی روی داده‌های خام و داده‌انکود شده تفاوت زیادی ندارند. درواقع انکود کردن داده‌توانسته تا حد بسیار خوبی ویژگی‌های اصلی داده‌ها را حفظ کند. اگرچه در بعضی کلاس‌ها ممکن است تفاوتی دیده شود که این به این دلیل است که الگوریتم K-means به فاصله داده‌ها حساس است و وقتی داده‌ها را encode میکنیم ممکن است فاصله‌های بعضی ویژگی‌ها در فضای encode شده تغییر کند. حال با استفاده از الگوریتم t-sne داده‌ها را به فضای دو بعدی می‌بریم و رنگ‌آمیزی میکنیم:



شکل ۳۵: 2D test dataset(without Encoding)

در اینجا هم مشخص است که نمودار با قبل و بعد از انکود کردن داده ها تفاوت زیادی نکرده است. به صورت کلی و با توجه به نتایج بدست آمده در این چند صفحه می توان به این نتیجه رسید که انکود کردن داده ها باعث نشده اطلاعات زیاد از دست برود و اطلاعات مهم داده ها همچنان حفظ شده است. مقدار ۱۰ در گلوگاه این AutoEncoder با آزمون و خطا بدست آمده و برای مقادیر کمتر از ۱۰ مدل به خوبی یاد نمی گرفت و بخش زیادی از اطلاعات مهم از دست می رفت.

۴-۴-۶ ارزیابی کیفیت خوشه بندی

با توجه به خواسته صورت سوال شاخص های AMI و ARI را در دو تابع جداگانه پیاده سازی کردیم و برای داده های encode شده و داده های خام این مقادیر زیر را بدست آورده ایم:

جدول ۱۷: Comparison of clustering metrics for raw and encoded test datasets

Raw Test Dataset	Encoded Test Dataset	Metric
0.510	0.502	AMI
0.370	0.357	ARI

که حداقل امتیاز های خواسته شده را ارضا می کند.